

Taller VPN

Created @28 de octubre de 2025 16:26

Pasos instalación:

1. Descargar el disco virtual

2. Lo añadimos a la máquina virtual. Para ello vamos a la maquina en la que lo queremos añadir y pulsamos en *Edit virtual machine settings*. En la pestaña de *Hardware* pulsamos en *Add*. Escogemos *Hard Disk* > *SCSI* > *Use an existing virtual disk* y esocgemos el fichero que descargamos.

3. Entramos en la máquina virtual

4. Descargamos desde la propia máquina el script `strongswan_install.sh` . Le damos permisos (`chmod +x strongswan_install.sh`) y lo ejecutamos.

5. Vamos a la carpeta `/srv/strongswan-testing/testing` e iniciamos el entorno de pruebas `./start-testing` , tiene que salir todo OK.

```
root@omnetvm2024:/srv/strongswan-testing/testing# sudo ./start-testing
Starting test environment
[ ok ] Deploying kernel linux-5.19.11
[ ok ] Deploying /srv/strongswan-testing/build/shared/bullseye as hostfs
[ ok ] Deploying /srv/strongswan-testing/testresults as hostfs
[ ok ] Network vnet1
[ ok ] Network vnet2
[ ok ] Network vnet3
[ ok ] Guest alice
[ ok ] Guest bob
[ ok ] Guest carol
[ ok ] Guest dave
[ ok ] Guest moon
[ ok ] Guest sun
[ ok ] Guest venus
[ ok ] Guest winnetou
```

Para pararlo usamos `./stop-testing`

Cuestiones práctica

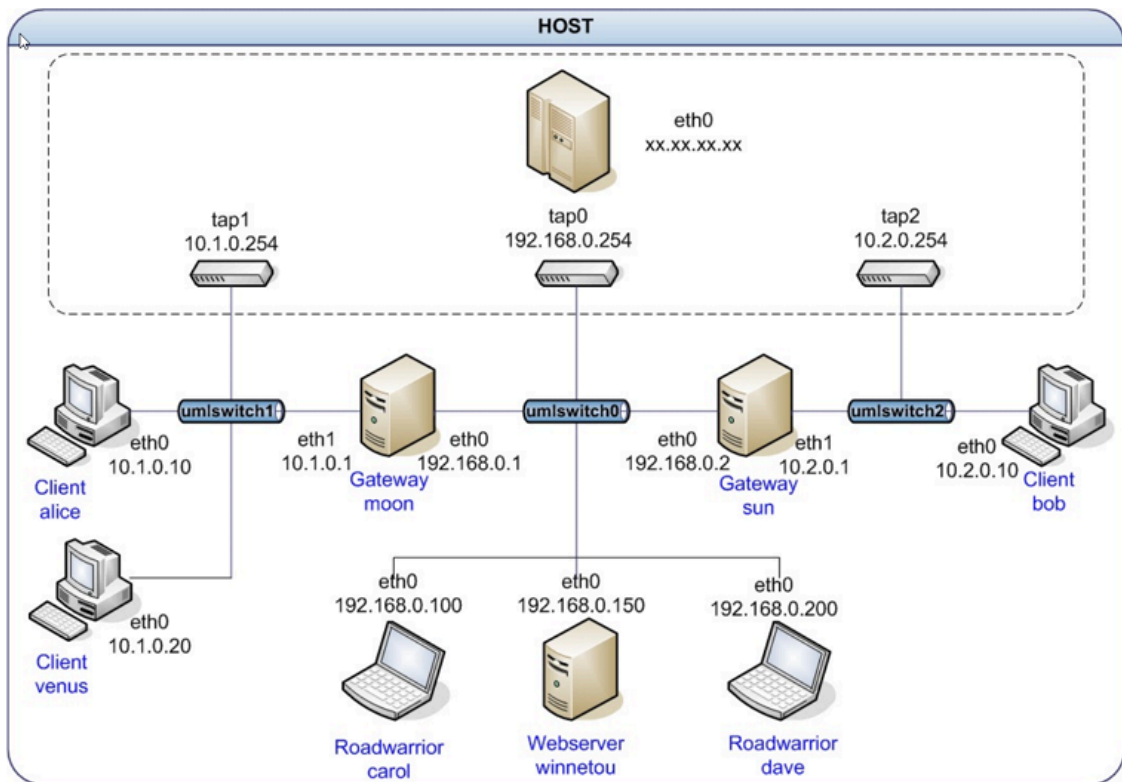


Figura 1 Diagrama de la red del entorno de pruebas de strongSwan.

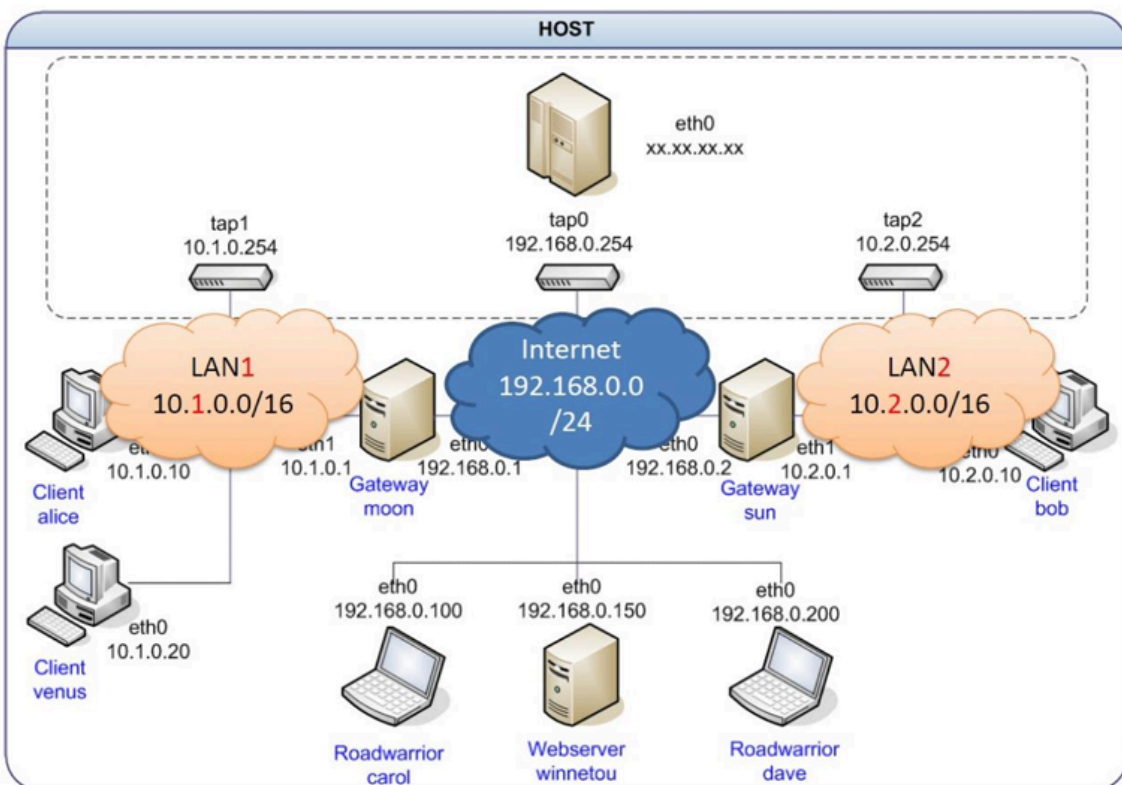


Figura 2 Subredes de la red del entorno de pruebas de strongSwan.

1.2 Pruebas básicas

Muestra las tablas de enrutamiento de los equipos alice, bob, moon, sun, carol y winnetou.

Para mostrar las tablas de enrutamiento debemos acceder a cada equipo y ejecutar el comando `ip route`.

- Nota: para entrar a cada ordenador debemos abrir una nueva pestaña en tu terminal y ejecutar `ssh <nombre_de_la_maquina>` (por ejemplo: `ssh ip_alice`).

Tabla de enrutamiento de Alice:

```
alice:~# ip route
default via 10.1.0.1 dev eth0 onlink
10.1.0.0/16 dev eth0 proto kernel scope link src 10.1.0.10
10.8.0.1 via 10.8.0.5 dev tun0
10.8.0.5 dev tun0 proto kernel scope link src 10.8.0.6
```

Tabla de enrutamiento de Bob:

```
bob:~# ip route
default via 10.2.0.1 dev eth0 onlink
10.2.0.0/16 dev eth0 proto kernel scope link src 10.2.0.10
```

Tabla de enrutamiento de Moon:

```
moon:~# ip route
default via 192.168.0.254 dev eth0 onlink
10.1.0.0/16 dev eth1 proto kernel scope link src 10.1.0.1
10.8.0.0/24 via 10.8.0.2 dev tun0
10.8.0.2 dev tun0 proto kernel scope link src 10.8.0.1
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.1
moon:~#
```

Tabla de enrutamiento de Sun:

```
sun:~# ip route
default via 192.168.0.254 dev eth0 onlink
10.2.0.0/16 dev eth1 proto kernel scope link src 10.2.0.1
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.2
sun:~#
```

Tabla de enrutamiento de Carol:

```
carol:~# ip route
default via 192.168.0.254 dev eth0 onlink
10.8.0.1 via 10.8.0.9 dev tun0
10.8.0.9 dev tun0 proto kernel scope link src 10.8.0.10
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.100
carol:~#
```

Tabla de enrutamiento de Winnetou:

```
winnetou:~# ip route
default via 192.168.0.254 dev eth0 onlink
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.150
winnetou:~#
```

A la vista de las tablas de enrutamiento y comprobando mediante ping desde cada uno de los equipos:

¿Es posible alcanzar bob desde alicé? ¿Desde moon? ¿Desde carol?

- **Alicé → Bob** : Alicé (10.1.0.10) envía el tráfico a su puerta de enlace predeterminada, que es **moon** (10.1.0.1). Sin embargo, **moon** no tiene una ruta para llegar a la red **LAN2** (10.2.0.0/16), por lo que el paquete no llega nunca a **Bob**.

```
alice:~# ping 10.2.0.10
PING 10.2.0.10 (10.2.0.10) 56(84) bytes of data.
From 10.1.0.254 icmp_seq=1 Destination Port Unreachable
From 10.1.0.254 icmp_seq=2 Destination Port Unreachable
From 10.1.0.254 icmp_seq=3 Destination Port Unreachable
From 10.1.0.254 icmp_seq=4 Destination Port Unreachable
From 10.1.0.254 icmp_seq=5 Destination Port Unreachable
^C
--- 10.2.0.10 ping statistics ---
5 packets transmitted, 0 received, +5 errors, 100% packet loss, time 4008ms
```

- **Moon → Bob**: Aunque **Moon** está en la misma red que **Sun** (el gateway de **Bob**), **Moon** no sabe que para llegar a la dirección privada de **Bob** (10.2.0.10) tiene que enviar el paquete a través de la IP de **Sun** (192.168.0.2). Por esta razón el paquete no llega nunca a **Bob**.

```
moon:~# ping 10.2.0.10
PING 10.2.0.10 (10.2.0.10) 56(84) bytes of data.
From 192.168.0.254 icmp_seq=1 Destination Port Unreachable
From 192.168.0.254 icmp_seq=2 Destination Port Unreachable
From 192.168.0.254 icmp_seq=3 Destination Port Unreachable
From 192.168.0.254 icmp_seq=4 Destination Port Unreachable
From 192.168.0.254 icmp_seq=5 Destination Port Unreachable
^C
--- 10.2.0.10 ping statistics ---
5 packets transmitted, 0 received, +5 errors, 100% packet loss, time 4074ms
```

- **Carol → Bob**: Al igual que el resto, **Carol** no tiene ninguna ruta para llegar a la red de **Bob** por lo que el paquete nunca va a llegar.

```
carol:~# ping 10.2.0.10
PING 10.2.0.10 (10.2.0.10) 56(84) bytes of data.
From 192.168.0.254 icmp_seq=1 Destination Port Unreachable
From 192.168.0.254 icmp_seq=2 Destination Port Unreachable
From 192.168.0.254 icmp_seq=3 Destination Port Unreachable
^C
--- 10.2.0.10 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2005ms
```

¿Es posible alcanzar carol desde moon? ¿Desde winnetou? ¿Desde bob?

- **Moon → Carol:** **Sí hay conectividad.** Ambos equipos están conectados al mismo switch (`umiswitch0`) y pertenecen a la misma subred (192.168.0.0/24).

```
moon:~# ping 192.168.0.100
PING 192.168.0.100 (192.168.0.100) 56(84) bytes of data.
64 bytes from 192.168.0.100: icmp_seq=1 ttl=64 time=0.758 ms
64 bytes from 192.168.0.100: icmp_seq=2 ttl=64 time=0.700 ms
64 bytes from 192.168.0.100: icmp_seq=3 ttl=64 time=0.794 ms
64 bytes from 192.168.0.100: icmp_seq=4 ttl=64 time=0.933 ms
^C
--- 192.168.0.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.700/0.796/0.933/0.085 ms
```

- **Winnetou → Carol:** **Sí hay conectividad.** Al igual que la anterior, el servidor Winnetou (192.168.0.150) y Carol (192.168.0.100) pertenecen al mismo segmento de red.

```
winnetou:~# ping 192.168.0.100
PING 192.168.0.100 (192.168.0.100) 56(84) bytes of data.
64 bytes from 192.168.0.100: icmp_seq=1 ttl=64 time=2.24 ms
64 bytes from 192.168.0.100: icmp_seq=2 ttl=64 time=1.10 ms
^C
--- 192.168.0.100 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 1.099/1.671/2.244/0.572 ms
```

- **Bob → Carol:** **Bob** envía el paquete a su gateway por defecto, **Sun**, y este lo reenvía a **Carol**. El problema surge cuando **Carol** intenta responder al paquete: como no sabe a dónde enviar la respuesta (No tiene en su tabla de enrutamiento ninguna ruta para llegar a la red 10.2.0.0/16), la envía a su gateway por defecto, por lo que el paquete nunca llega a **Bob**.

```
bob:~# ping 192.168.0.100
PING 192.168.0.100 (192.168.0.100) 56(84) bytes of data.
^C
--- 192.168.0.100 ping statistics ---
23 packets transmitted, 0 received, 100% packet loss, time 22509ms
```

¿Es posible alcanzar alice desde sun? ¿Desde winnetou? ¿Desde moon?

- **Sun → Alice: No hay conectividad**, **Sun** intenta enviar el paquete pero no tiene ninguna ruta en su tabla que le diga que para poder llegar a la red 10.1.0.0./16 tiene que enviarlo a través de **Moon**, entonces lo envía a través de su gateway por defecto (192.168.0.254) por ende el paquete no puede llegar a **Alice**.

```
sun:~# ping 10.1.0.10
PING 10.1.0.10 (10.1.0.10) 56(84) bytes of data.
From 192.168.0.254 icmp_seq=1 Destination Port Unreachable
From 192.168.0.254 icmp_seq=2 Destination Port Unreachable
^C
--- 10.1.0.10 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1024ms
```

- **Winnetou → Alice: No hay conectividad**, **Winnetou** intenta enviar el paquete pero no tiene ninguna ruta en su tabla que le diga que para poder llegar a la red 10.1.0.0./16 tiene que enviarlo a través de **Moon**, entonces lo envía a través de su gateway por defecto (192.168.0.254) por ende el paquete no puede llegar a **Alice**.

```
winnetou:~# ping 10.1.0.10
PING 10.1.0.10 (10.1.0.10) 56(84) bytes of data.
From 192.168.0.254 icmp_seq=1 Destination Port Unreachable
From 192.168.0.254 icmp_seq=2 Destination Port Unreachable
From 192.168.0.254 icmp_seq=3 Destination Port Unreachable
^C
--- 10.1.0.10 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2005ms
```

- **Moon → Alice: Si hay conectividad**, **Moon** es el gateway de la **LAN1**. Tiene una interfaz física (**eth1**) conectada directamente a esa red con la IP 10.1.0.1, por lo que conoce el camino hacia **alice** (10.1.0.10).

```
moon:~# ping 10.1.0.10
PING 10.1.0.10 (10.1.0.10) 56(84) bytes of data.
64 bytes from 10.1.0.10: icmp_seq=1 ttl=64 time=0.932 ms
64 bytes from 10.1.0.10: icmp_seq=2 ttl=64 time=0.788 ms
64 bytes from 10.1.0.10: icmp_seq=3 ttl=64 time=0.697 ms
64 bytes from 10.1.0.10: icmp_seq=4 ttl=64 time=0.659 ms
^C
--- 10.1.0.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.659/0.769/0.932/0.105 ms
```

¿Cuáles son las direcciones del equipo omnet6-vm? ¿Qué direcciones del equipo anfitrión es posible alcanzar desde cada equipo?

Para poder ver las direcciones IP del equipo anfitrión, se utiliza el comando `ip -br address show`.

```

root@omnetvm2024:/srv/strongswan-testing/testing# ip -br address show
lo                UNKNOWN      127.0.0.1/8 ::1/128
ens192            UP           192.168.228.130/24 fe80::44e5:1531:994a:2b04/64
virbr0           DOWN
test-br0          UP           192.168.0.254/24
test-br1          UP           10.1.0.254/16
test-br2          UP           10.2.0.254/16
alice-eth0        UNKNOWN      fe80::fc54:ff:fe9a:e2de/64
alice-eth1        UNKNOWN      fe80::fc54:ff:fe3b:cd7/64
bob-eth0          UNKNOWN      fe80::fc54:ff:fe40:856b/64
carol-eth0        UNKNOWN      fe80::fc54:ff:feae:f1f8/64
dave-eth0         UNKNOWN      fe80::fc54:ff:feb9:15a9/64
moon-eth1         UNKNOWN      fe80::fc54:ff:fe43:e335/64
moon-eth0         UNKNOWN      fe80::fc54:ff:fec7:b8b0/64
sun-eth0          UNKNOWN      fe80::fc54:ff:fe77:43ea/64
sun-eth1          UNKNOWN      fe80::fc54:ff:fe0f:97db/64
venus-eth0        UNKNOWN      fe80::fc54:ff:fe69:d380/64
winnetou-eth0     UNKNOWN      fe80::fc54:ff:fe4b:23fa/64
root@omnetvm2024:/srv/strongswan-testing/testing#

```

- **test-br0** **UP** **192.168.0.254/24** : Dirección del host en la red de **Internet** del escenario.
- **test-br1** **UP** **10.1.0.254/16** : Dirección del host en la **LAN1**.
- **test-br2** **UP** **10.2.0.254/16** : Dirección del host en la **LAN2**.

También podríamos verlo a través del comando `ip route` que nos muestra la tabla de enrutamiento del equipo anfitrión.

- **10.1.0.0/16 dev test-br1** : Cualquier tráfico para la red 10.1.x.x se envía por la interfaz **test-br1**. La IP del host en ese bridge es **10.1.0.254**.
- **10.2.0.0/16 dev test-br2** : Similar a la anterior, pero para la red 10.2.x.x a través de **test-br2**. La IP del host es **10.2.0.254**.
- **192.168.0.0/24 dev test-br0** : Tráfico para la red local 192.168.0.x a través de **test-br0**. La IP del host es **192.168.0.254**.

Cada equipo solo puede alcanzar la IP del host que esta en su mismo segmento de red:

Equipo(s) de origen	Segmento de Red	IP del Host alcanzable
alice, venus	LAN 1 (10.1.0.0/16)	10.1.0.254
moon, sun, carol, winnetou, dave	Internet (192.168.0.0/24)	192.168.0.254
bob	LAN 2 (10.2.0.0/16)	10.2.0.254

1.3 Permitir la conectividad entre equipos y redes

Comunicación entre dos equipos

Para permitir que **Carol** se conecte a **Alice** podemos añadir una ruta estática en su tabla de enrutamiento para que sepa como llegar al destino. Para ello podemos usar el comando `ip route add 10.1.0.10 via 192.168.0.1` (`ip route add <destino> via <gateway>`). Ahora vamos a realizar una serie de pruebas:

- **Ping Carol → Alice:** Este ping si que realiza de forma correcta ya que **Carol** tiene en su tabla de enrutamiento (Gracias a la ruta que acabamos de añadir) el camino para llegar a **Alice**

10.1.0.10.

```
carol:~# ip route add 10.1.0.10 via 192.168.0.1
carol:~# ping 10.1.0.10
PING 10.1.0.10 (10.1.0.10) 56(84) bytes of data.
64 bytes from 10.1.0.10: icmp_seq=1 ttl=63 time=1.80 ms
64 bytes from 10.1.0.10: icmp_seq=2 ttl=63 time=1.41 ms
64 bytes from 10.1.0.10: icmp_seq=3 ttl=63 time=1.56 ms
^C
--- 10.1.0.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.411/1.592/1.804/0.161 ms
```

- **Ping Alice → Carol:** Este ping también se realiza de forma correcta. Antes el problema que teníamos al enviar un ping a **Carol** desde **Alice** era que, **Alice** al no saber una ruta para llegar a **Carol** lo enviaba a su gateway por defecto, **Moon**, y esta se lo reenviaba a **Carol** (Ya que están en el mismo segmento de red). El problema estaba en que **Carol** no sabía hacia donde enviar la respuesta y la enviaba a su gateway por defecto, que era internet, haciendo que la respuesta nunca llegase. Ahora como **Carol** tiene la ruta para llegar a **Alice** sabe hacia donde encaminar dicha respuesta, por lo que el ping funciona correctamente.
- **Ping Carol → Venus:** Este ping no funciona ya que **Carol** solo sabe la ruta para llegar a **Alice** y no a **Venus**. Para poder solucionar esto, tendríamos que indicarle a **Carol** como llegar a la **LAN1** y no solamente a un host de dicha LAN.

Ahora vamos a borrar esta ruta (`ip route del 10.1.0.10`) y a añadir una para permitir la conectividad de **Carol** con la red 10.1.0.0/16. Para ello utilizamos el comando : `ip route add 10.1.0.0/16 via 192.168.0.1` . Podemos ver que se ha añadido correctamente revisando su tabla de enrutamiento:

```
carol:~# ip route add 10.1.0.0/16 via 192.168.0.1
carol:~# ip route show
default via 192.168.0.254 dev eth0 onlink
10.1.0.0/16 via 192.168.0.1 dev eth0
10.8.0.1 via 10.8.0.9 dev tun0
10.8.0.9 dev tun0 proto kernel scope link src 10.8.0.10
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.100
```

Ahora vamos a probar lo siguiente:

- **Ping Carol → Alice:** Al igual que con la ruta anterior este ping funciona correctamente.
- **Ping Alice → Carol:** Al igual que con la ruta anterior este ping funciona correctamente.
- **Ping Carol → Venus:** Este ping funciona ya que ahora **Carol** conoce la ruta para llegar a la red **LAN1** (10.1.0.0/16) .

Comunicacion entre las dos LAN

Si queremos permitir la comunicación entre las subredes **LAN1** y **LAN2** lo que debemos hacer es añadir una ruta en los gateways **Moon** y **Sun**.

```
moon$ ip route add 10.2.0.0/16 via 192.168.0.2
```

```
moon:~# ip route add 10.2.0.0/16 via 192.168.0.2
moon:~# ip route
default via 192.168.0.254 dev eth0 onlink
10.1.0.0/16 dev eth1 proto kernel scope link src 10.1.0.1
10.2.0.0/16 via 192.168.0.2 dev eth0
10.8.0.0/24 via 10.8.0.2 dev tun0
10.8.0.2 dev tun0 proto kernel scope link src 10.8.0.1
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.1
```

```
sun$ ip route add 10.1.0.0/16 via 192.168.0.1
```

```
sun:~# ip route add 10.1.0.0/16 via 192.168.0.1
sun:~# ip route
default via 192.168.0.254 dev eth0 onlink
10.1.0.0/16 via 192.168.0.1 dev eth0
10.2.0.0/16 dev eth1 proto kernel scope link src 10.2.0.1
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.2
```

Ahora podemos comunicarnos entre cualquier host de la **LAN1** y **LAN2**.

- Alice ↔ Bob
- Venus ↔ Bob

2. Conceptos básicos de cortafuegos

Muestra la tabla de enrutamiento y las reglas de filtrado del cortafuegos del equipo anfitrión:

Para mostrar la tabla de enrutamiento podemos usar el comando anteriormente usado o usar `route -n` el cual produce la siguiente salida:

```
root@omnetvm2024:/srv/strongswan-testing/testing# route -n
Kernel IP routing table
Destination    Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0        192.168.228.2  0.0.0.0         UG    100    0      0 ens192
10.1.0.0       0.0.0.0        255.255.0.0     U      0      0      0 test-br1
10.2.0.0       0.0.0.0        255.255.0.0     U      0      0      0 test-br2
192.168.0.0    0.0.0.0        255.255.255.0   U      0      0      0 test-br0
192.168.122.0  0.0.0.0        255.255.255.0   U      0      0      0 virbr0
192.168.228.0  0.0.0.0        255.255.255.0   U     100    0      0 ens192
```

Para mostrar las reglas de filtrado se usa `iptables -L -v` (Aunque también se puede usar `iptables -S -v` para verlas en otro formato).

```

root@omnetvm2024:/srv/strongswan-testing/testing# iptables -L -v
Chain INPUT (policy ACCEPT 338K packets, 64M bytes)
pkts bytes target prot opt in out source destination
338K 64M LIBVIRT_INP all -- any any anywhere anywhere

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
96 8064 LIBVIRT_FWX all -- any any anywhere anywhere
96 8064 LIBVIRT_FWI all -- any any anywhere anywhere
5 420 LIBVIRT_FWO all -- any any anywhere anywhere

Chain OUTPUT (policy ACCEPT 337K packets, 26M bytes)
pkts bytes target prot opt in out source destination
337K 26M LIBVIRT_OUT all -- any any anywhere anywhere

Chain LIBVIRT_FWI (1 references)
pkts bytes target prot opt in out source destination
0 0 ACCEPT all -- lo test-br2 anywhere 10.2.0.0/16
60 5040 REJECT all -- any test-br2 anywhere anywhere reject-with icmp-port-unreachable
0 0 ACCEPT all -- lo test-br1 anywhere 10.1.0.0/16
31 2604 REJECT all -- any test-br1 anywhere anywhere reject-with icmp-port-unreachable
0 0 ACCEPT all -- lo test-br0 anywhere 192.168.0.0/24
0 0 REJECT all -- any test-br0 anywhere anywhere reject-with icmp-port-unreachable
0 0 ACCEPT all -- any virbr0 anywhere 192.168.122.0/24 ctstate RELATED,ESTABLISHED
0 0 REJECT all -- any virbr0 anywhere anywhere reject-with icmp-port-unreachable

```

```

Chain LIBVIRT_FWX (1 references)
pkts bytes target prot opt in out source destination
0 0 ACCEPT all -- test-br2 test-br2 anywhere anywhere
0 0 ACCEPT all -- test-br1 test-br1 anywhere anywhere
0 0 ACCEPT all -- test-br0 test-br0 anywhere anywhere
0 0 ACCEPT all -- virbr0 virbr0 anywhere anywhere

Chain LIBVIRT_INP (1 references)
pkts bytes target prot opt in out source destination
0 0 ACCEPT udp -- test-br2 any anywhere anywhere udp dpt:domain
0 0 ACCEPT tcp -- test-br2 any anywhere anywhere tcp dpt:domain
0 0 ACCEPT udp -- test-br2 any anywhere anywhere udp dpt:bootps
0 0 ACCEPT tcp -- test-br2 any anywhere anywhere tcp dpt:67
0 0 ACCEPT udp -- test-br1 any anywhere anywhere udp dpt:domain
0 0 ACCEPT tcp -- test-br1 any anywhere anywhere tcp dpt:domain
0 0 ACCEPT udp -- test-br1 any anywhere anywhere udp dpt:bootps
0 0 ACCEPT tcp -- test-br1 any anywhere anywhere tcp dpt:67
0 0 ACCEPT udp -- test-br0 any anywhere anywhere udp dpt:domain
0 0 ACCEPT tcp -- test-br0 any anywhere anywhere tcp dpt:domain
0 0 ACCEPT udp -- test-br0 any anywhere anywhere udp dpt:bootps
0 0 ACCEPT tcp -- test-br0 any anywhere anywhere tcp dpt:67
0 0 ACCEPT udp -- virbr0 any anywhere anywhere udp dpt:domain
0 0 ACCEPT tcp -- virbr0 any anywhere anywhere tcp dpt:domain
0 0 ACCEPT udp -- virbr0 any anywhere anywhere udp dpt:bootps
0 0 ACCEPT tcp -- virbr0 any anywhere anywhere tcp dpt:67

```

```

Chain LIBVIRT_OUT (1 references)
pkts bytes target prot opt in out source destination
0 0 ACCEPT udp -- any test-br2 anywhere anywhere udp dpt:domain
0 0 ACCEPT tcp -- any test-br2 anywhere anywhere tcp dpt:domain
0 0 ACCEPT udp -- any test-br2 anywhere anywhere udp dpt:bootpc
0 0 ACCEPT tcp -- any test-br2 anywhere anywhere tcp dpt:68
0 0 ACCEPT udp -- any test-br1 anywhere anywhere udp dpt:domain
0 0 ACCEPT tcp -- any test-br1 anywhere anywhere tcp dpt:domain
0 0 ACCEPT udp -- any test-br1 anywhere anywhere udp dpt:bootpc
0 0 ACCEPT tcp -- any test-br1 anywhere anywhere tcp dpt:68
0 0 ACCEPT udp -- any test-br0 anywhere anywhere udp dpt:domain
0 0 ACCEPT tcp -- any test-br0 anywhere anywhere tcp dpt:domain
0 0 ACCEPT udp -- any test-br0 anywhere anywhere udp dpt:bootpc
0 0 ACCEPT tcp -- any test-br0 anywhere anywhere tcp dpt:68
0 0 ACCEPT udp -- any virbr0 anywhere anywhere udp dpt:domain
0 0 ACCEPT tcp -- any virbr0 anywhere anywhere tcp dpt:domain
0 0 ACCEPT udp -- any virbr0 anywhere anywhere udp dpt:bootpc
0 0 ACCEPT tcp -- any virbr0 anywhere anywhere tcp dpt:68

```

La salida del comando muestra las reglas de filtrado del anfitrión, que están organizadas en cadenas para controlar el flujo de datos. En este escenario virtual, la gestión se delega en subcadenas específicas de **libvirt** para organizar el tráfico de las máquinas virtuales:

- **LIBVIRT_INP (Input):** Controlan el tráfico con destino al propio equipo anfitrión. Permiten servicios de infraestructura esenciales como **DNS** (puerto 53) y **DHCP** (puertos 67/68), asegurando que los equipos virtuales puedan obtener una dirección IP y resolver nombres.
- **LIBVIRT_OUT (Output):** Gestionan el tráfico que sale desde el anfitrión hacia las redes virtuales, permitiendo que las respuestas de los servicios anteriormente mencionados lleguen

correctamente a las máquinas.

- **LIBVIRT_FWX (Forwarding Exchange)**: Permite la comunicación **interna** dentro de una misma subred. Al aceptar paquetes donde la interfaz de entrada y salida es la misma (ej. de `test-br1` a `test-br1`), garantiza que equipos como Alice y Venus puedan hablar entre sí.
- **LIBVIRT_FWI (Forwarding Isolation)**: Es la cadena responsable del **aislamiento de red**. Contiene reglas **REJECT** que bloquean explícitamente el paso de paquetes entre diferentes interfaces de puente. Este es el motivo por el cual, aunque existan rutas, el tráfico entre subredes (como de LAN1 a LAN2) es rechazado por el host con un mensaje de "puerto inalcanzable".

Muestra la tabla nat del cortafuegos del equipo anfitrión:

Para mostrar la tabla nat del firewall usamos el comando `iptables -t nat -L -v`

```
root@omnetvm2024:/srv/strongswan-testing/testing# iptables -t nat -L -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
Chain POSTROUTING (policy ACCEPT 1466 packets, 111K bytes)
 pkts bytes target    prot opt in     out     source    destination
 1466 111K LIBVIRT_PRT all  --  any    any    anywhere  anywhere
Chain LIBVIRT_PRT (1 references)
 pkts bytes target    prot opt in     out     source    destination
 0      0 RETURN  all  --  any    any    192.168.122.0/24  base-address.mcast.net/24
 0      0 RETURN  all  --  any    any    192.168.122.0/24  255.255.255.255
 0      0 MASQUERADE tcp  --  any    any    192.168.122.0/24  !192.168.122.0/24    masq ports: 1024-65535
 0      0 MASQUERADE udp  --  any    any    192.168.122.0/24  !192.168.122.0/24    masq ports: 1024-65535
 0      0 MASQUERADE all  --  any    any    192.168.122.0/24  !192.168.122.0/24
```

Explicación de los campos:

- **pkts / bytes**: Son los contadores. Indican cuántos paquetes y cuántos datos (en bytes) han pasado por esa regla específica.
- **target**: Es la **acción** que se toma.
 - **RETURN**: Funciona como una **excepción anticipada**. Detiene el procesamiento de la lista de reglas para paquetes específicos (como el tráfico de red local o gestión), evitando que lleguen a ser procesados por las reglas de enmascaramiento (**MASQUERADE**) que vienen después.
 - **MASQUERADE**: Es una forma de NAT dinámica. "Disfraza" la IP del equipo interno con la IP de salida del anfitrión.
- **prot**: El protocolo al que se aplica (tcp, udp o all).
- **in / out**: La interfaz de red por la que entra o sale el paquete.
- **source**: La dirección IP de origen.
- **destination**: La dirección IP de destino. El símbolo **!** delante de una IP significa "todo lo que **NO** sea esta red".

¿Qué reglas iptables aíslan el tráfico de cada una de las tres redes?

El aislamiento entre las subredes del escenario se gestiona en el escenario por el anfitrión a través de la cadena **LIBVIRT_FWI** (Forward Isolation). Aunque el anfitrión tiene rutas hacia todas las redes, estas reglas actúan como un filtro que rechaza el tránsito entre los distintos puentes virtuales.

Las reglas que aíslan el tráfico son las que tienen la acción **REJECT** dentro de la cadena **LIBVIRT_FWI**.

1. **Aislamiento de la LAN2 (**test-br2**)**: Esta regla bloquea el tráfico que intenta entrar en la red de Bob (10.2.0.0/16) desde cualquier otra interfaz.

```
pkts bytes target prot opt in out source destination
60 5040 REJECT all -- any test-br2 anywhere anywhere reject-with icmp-port-unreachable
```

2. **Aislamiento de la LAN1 (**test-br2**)**: Cumple el mismo objetivo que la regla anterior pero para la red de Alice y Venus (10.1.0.0/16)

```
31 2604 REJECT all -- any test-br1 anywhere anywhere reject-with icmp-port-unreachable
```

Igual que la anterior pero para la LAN1

3. **Aislamiento de la red Internet (**virbr0**)**: Bloquea el tráfico que intenta entrar en la red intermedia desde cualquier otra interfaz.

```
0 0 REJECT all -- any virbr0 anywhere anywhere reject-with icmp-port-unreachable
```

Prueba a añadir como primera regla de la cadena FORWARD un “aceptar todo”, y comprueba si cambia en algo la conectividad entre los equipos de distintas redes.

La regla a añadir es la siguiente:

```
iptables -I FORWARD 1 -j ACCEPT
```

Lo que estamos haciendo es añadir en la posición 1 de la cadena **FORWARD**, que acepte todo el tráfico entre interfaces antes de que las reglas de **LIBVIRT_FWI** se apliquen.

Esto provoca que se anule el aislamiento que había entre las redes 10.1.0.0/16, 10.2.0.0/16 y 192.168.0.0/24. Ahora es posible hacer pings exitosos entre cualquier equipo de las tres subredes.

Para eliminar la regla anterior usamos: `iptables -D FORWARD 1`

2.1 Source NAT (MASQUERADE)

Habilita SNAT (tipo MASQUERADE) en moon para permitir la conexión desde la red 10.1.0.0/16 a la red 192.168.0.0/24:

¿Por qué MASQUERADE en moon?

Las máquinas de la LAN 1 (como Alice) tienen direcciones IP privadas (10.1.0.0/16). Carol, que está en la red 192.168.0.0/24, no sabe cómo llegar a esas IPs privadas.

- **Sin NAT**: Alice le envía un paquete a Carol. Carol lo recibe, pero al intentar responder a una IP 10.1.x.x, el paquete se pierde porque Carol no tiene esa ruta.
- **Con MASQUERADE**: Cuando el paquete de Alice pasa por **moon**, este cambia la IP de origen (Alice) por la suya propia (192.168.0.1 en su interfaz eth0). Carol cree que habla con **moon** y responde. Al volver el paquete, **moon** sabe que es para Alice y se lo entrega.

Para activarlo usamos el comando `iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE` en moon. Para ver que se ha aplicado correctamente podemos usar el comando `iptables -t nat -L -v`. La salida del comando nos muestra :

```
moon:~# iptables -t nat -L -v
Chain PREROUTING (policy ACCEPT 3 packets, 144 bytes)
 pkts bytes target      prot opt in     out     source destination
Chain INPUT (policy ACCEPT 3 packets, 144 bytes)
 pkts bytes target      prot opt in     out     source destination
Chain OUTPUT (policy ACCEPT 1 packets, 120 bytes)
 pkts bytes target      prot opt in     out     source destination
Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target      prot opt in     out     source destination
    1  120 MASQUERADE all  --  any    eth0    anywhere anywhere
```

Explicación campos:

- **Cadena POSTROUTING:** Indica que la regla se aplica a los paquetes en el último paso posible, justo antes de que abandonen el equipo hacia la red externa.
- **Target MASQUERADE:** Confirma que se está utilizando el enmascaramiento dinámico para traducir las direcciones IP de origen de la red interna.
- **Interfaz de salida (out eth0):** La regla está restringida exclusivamente a la interfaz **eth0**, que es la que conecta con la red 192.168.0.0/24 (donde está Carol).
- **Origen y Destino (anywhere):** Al no especificar redes concretas, permite que cualquier equipo de la LAN 1 pueda salir hacia cualquier destino externo a través de esta interfaz.
- **Contadores de tráfico:** Se puede observar que ya se ha procesado **1 paquete** (120 bytes), lo que demuestra que el NAT está interceptando y traduciendo tráfico real en este momento.

Comprueba la conectividad alice --> carol y la ausencia de conectividad carol -> alice, primero mediante ping y traceroute, y luego mediante ssh. Una vez establecida la conexión ssh alice --> carol, muestra las conexiones en carol mediante `netstat -4 an`.

- **Conectividad Alice → Carol:** Sí hay conectividad, porque la IP del paquete de **Alice** se **enmascara con la IP de Moon**. **Carol** responde creyendo que el paquete va dirigido a **Moon**, y **Moon** recibe la respuesta y se la reenvía a **Alice**.

Ping de Alice a Carol:

```
alice:~# ping 192.168.0.100
PING 192.168.0.100 (192.168.0.100) 56(84) bytes of data.
64 bytes from 192.168.0.100: icmp_seq=1 ttl=63 time=1.99 ms
64 bytes from 192.168.0.100: icmp_seq=2 ttl=63 time=1.56 ms
^C
--- 192.168.0.100 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.563/1.774/1.986/0.211 ms
```

Traceroute de Alice a Carol:

```

alice:~# traceroute 192.168.0.100
traceroute to 192.168.0.100 (192.168.0.100), 30 hops max, 60 byte packets
 1  moon1.strongswan.org (10.1.0.1)  1.718 ms  1.574 ms  1.954 ms
 2  carol.strongswan.org (192.168.0.100)  3.269 ms  3.312 ms  3.436 ms

```

SSH de Alice a Carol: Alice se conecta por **SSH** a Carol. Para ver las conexiones activas en **Carol** y comprobar desde qué dirección IP llega la conexión SSH, se usa el comando `ss -4an`

```

alice:~# ssh 192.168.0.100
Warning: Permanently added '192.168.0.100' (ECDSA) to the list of known hosts.
carol:~# ss -4an
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port
Process
udp    UNCONN  0      0      0.0.0.0:50249    0.0.0.0:*
tcp    LISTEN  0      128     0.0.0.0:22      0.0.0.0:*
tcp    ESTAB   0      0  192.168.0.100:22  192.168.0.1:57264

```

La dirección de origen que registra Carol es la **192.168.0.1**, que pertenece a la interfaz de salida de **moon**, y no la IP real de Alice (10.1.0.10). Esto demuestra que el tráfico ha sido enmascarado correctamente.

- **Conectividad Carol → Alice:** No hay conectividad ya que **Carol** no tiene ninguna ruta para llegar a **Alice**.

¿Cuál es la dirección IP origen de alicia que ve carol? ¿Y alicia de carol?

- **Carol de Alice:** Carol ve la dirección 192.168.0.1. Esta dirección no pertenece a Alice, sino a la interfaz externa de **moon**. Debido a la regla de enmascaramiento, la IP privada de Alice (10.1.0.10) queda oculta para los equipos de la red externa.
- **Alice de Carol:** Alice ve la dirección real de Carol, 192.168.0.100. Esto ocurre porque el tráfico saliente desde la LAN1 no modifica la dirección IP de destino (Carol), solo la de origen (Alice).

Nota→ Para eliminar la regla usamos: `iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE`

Efectúa un MASQUERADE desde moon que permita solo a alicia conectar con la red 192.168.0.0/24

Para hacer esto vamos a usar el comando `iptables -t nat -A POSTROUTING -o eth0 -s 10.1.0.10 -j MASQUERADE`.

```
moon:~# iptables -t nat -A POSTROUTING -o eth0 -s 10.1.0.10 -j MASQUERADE
moon:~# iptables -t nat -L -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target      prot opt in     out     source         destination
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target      prot opt in     out     source         destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target      prot opt in     out     source         destination
Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target      prot opt in     out     source         destination
    0      0 MASQUERADE  all  --  any    eth0    alice.strongswan.org  anywhere
```

A diferencia de la regla anterior que permitía la salida a toda la subred, el parámetro `-s 10.1.0.10` actúa como un filtro. Solo cuando el paquete proviene de la IP de **Alice**, el sistema realiza la traducción de direcciones (NAT). Ahora equipos como **Venus** saldrán con su IP privada original `10.1x.x`.

Si ahora eliminamos la regla con `iptables -t nat -D POSTROUTING -o eth0 -s 10.1.0.10 -j MASQUERADE` **Alice** pierde la conectividad con **Carol** (`192.168.0.100`).

Efectúa MASQUERADE desde moon que permita solo a alicé conectar con el servidor web en winnetou

Para esto vamos a usar el comando : `iptables -t nat -A POSTROUTING -o eth0 -s 10.1.0.10 -p tcp --dport 80 -j MASQUERADE` . Esta regla permite que únicamente el tráfico http generado por **Alice** (`10.1.0.10`) sea traducido al salir por `eth0` hacia la red `192.168.0.0/24`.

- **Conexión al servidor web:** Aunque la conexión ponga *Connection closed by remote host* (Esto se debe a que Winnetou no espera conexiones SSH en su puerto web) el mensaje confirma que **el paquete llegó a Winnetou** (`192.168.0.150`). Esto nos confirma que la regla de NAT puesta en **moon** funciona correctamente, de lo contrario el paquete no llegaría a **winnetou**.

```
alice:~# ssh -p 80 winnetou
kex_exchange_identification: Connection closed by remote host
Connection closed by 192.168.0.150 port 80
alice:~#
```

- **Ping de Alice → Winnetou:** No funciona ya que el comando `ping` utiliza el protocolo ICMP, no TCP. Al no haber una regla de NAT para ICMP, la comunicación hacia el exterior es imposible.
- **Conexión SSh Alice → Winnetou:** La conexión no se establece, aunque el origen sea Alice y el protocolo es TCP, el puerto 22 no coincide con la regla de NAT, por lo que el paquete no se traduce y la respuesta nunca llega a Alice.
- **Ping Venus → Winnetou:** Falla ya que el paquete desde **Winnetou** no puede volver a **Venus**.

Eliminamos la regla con : `iptables -t nat -D POSTROUTING -o eth0 -s 10.1.0.10 -p tcp --dport 80 -j MASQUERADE`

2.2 Destination NAT

Habilita DNAT para mapear el puerto 10022 de moon al puerto ssh de alicé

Para ello usamos en **moon** el siguiente comando: `iptables -t nat -A PREROUTING -p tcp --dport 10022 -j DNAT --to-destination 10.1.0.10:22` .

- `-t nat` : Indica que estamos trabajando en la tabla de **NAT** (Traducción de Direcciones de Red)
- `-A PREROUTING` : La regla se aplica en la cadena de **Pre-enrutamiento**. Esto ocurre justo cuando el paquete llega a la "puerta" de la pasarela **moon**, **antes** de que el sistema decida a dónde enviarlo.
- `-p tcp --dport 10022` : Es el filtro. Solo se activa si alguien intenta conectar usando el protocolo **TCP** al puerto **10022** de la pasarela.
- `-j DNAT` : El objetivo (*target*) es **Destination NAT**. Esto significa que vamos a cambiar la dirección IP de **destino** del paquete.
- `--to-destination 10.1.0.10:22` : Indica la nueva dirección y puerto. El paquete que originalmente iba dirigido a la IP de moon puerto 10022, será redirigido a la IP de **Alice** (10.1.0.10) al puerto **22** (SSH).

```
moon:~# iptables -t nat -L -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
    0      0 DNAT      tcp  --  any    any     anywhere  anywhere          tcp dpt:10022 to:10.1.0.10:22

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
moon:~#
```

Comprobamos ssh carol → alice vía DNAT: `carol$ ssh moon -p 10022` . Como podemos ver en la siguiente imagen, la conexión ssh al puerto 10022 de **moon** ha sido correctamente redirigida al servicio ssh de **Alice**.

```
carol:~# ssh moon -p 10022
Warning: Permanently added '[moon]:10022,[192.168.0.1]:10022' (ECDSA)
to the list of known hosts.
alice:~#
```

Eliminamos la regla usando: `iptables -t nat -D PREROUTING -p tcp --dport 10022 -j DNAT --to-destination 10.1.0.10:22`

2.2 Pruebas de velocidad

Realiza una prueba de velocidad alice --> moon

Para realizar la prueba se usa:

`moon$ iperf -s`

Este comando pone a **moon** en **modo servidor** (*server*).

- `s` : Indica que el equipo se queda "a la escucha". No envía datos por sí mismo, sino que abre un puerto (por defecto el 5001) y

`alice$ iperf -c moon -i 1`

Este comando pone a **Alice** en **modo cliente** (*client*) e inicia la prueba de estrés de red.

- `c moon` : Indica que Alice debe conectarse al equipo **moon**. Alice empezará a enviar una ráfaga de datos hacia la pasarela para ver qué tan rápida es la vía.

espera a que un cliente se conecte para empezar la prueba.

- **Función:** Actúa como el receptor del tráfico para medir cuánta información es capaz de recibir desde la red interna o externa.

- **i1 (interval):** Establece un intervalo de informe de **1 segundo**.

```
alice:~# iperf -c moon -i 1
-----
Client connecting to moon, TCP port 5001
TCP window size: 136 KByte (default)
-----
[ 3] local 10.1.0.10 port 41882 connected with 192.168.0.1 port 5001
[ ID] Interval           Transfer     Bandwidth
[ 3] 0.0000-1.0000 sec   279 MBytes  2.34 Gbits/sec
[ 3] 1.0000-2.0000 sec   312 MBytes  2.62 Gbits/sec
[ 3] 2.0000-3.0000 sec   309 MBytes  2.60 Gbits/sec
[ 3] 3.0000-4.0000 sec   298 MBytes  2.50 Gbits/sec
[ 3] 4.0000-5.0000 sec   304 MBytes  2.55 Gbits/sec
[ 3] 5.0000-6.0000 sec   296 MBytes  2.49 Gbits/sec
[ 3] 6.0000-7.0000 sec   308 MBytes  2.59 Gbits/sec
[ 3] 7.0000-8.0000 sec   318 MBytes  2.67 Gbits/sec
[ 3] 8.0000-9.0000 sec   323 MBytes  2.71 Gbits/sec
[ 3] 9.0000-10.0000 sec  319 MBytes  2.67 Gbits/sec
[ 3] 10.0000-10.0002 sec 384 KBytes 14.6 Gbits/sec
[ 3] 0.0000-10.0002 sec 3.00 GBytes 2.57 Gbits/sec
alice:~#
```

```
moon:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 192.168.0.1 port 5001 connected with 10.1.0.10 port 41882
[ ID] Interval           Transfer     Bandwidth
[ 4] 0.0000-10.0018 sec 3.00 GBytes 2.57 Gbits/sec
moon:~#
```

Como podemos observar en las imágenes se realizó una transferencia de **3.00 GBytes** con un ancho de banda medio de **2.57 Gbits/sec** . También podemos observar que la conexión se ha mantenido estable, manteniendo valores entre **2.34 y 2.71 Gbits/sec** durante la prueba.

Y moon --> alice

Usamos los mismos comandos que antes solo que intercambiamos a **alice** y **moon**.

```
moon$ iperf -c alice -i 1
```

Este comando pone a **Moon** en **modo cliente** (*client*) e inicia la prueba de estrés de red.

```
alice$ iperf -s
```

Este comando pone a **Alice** en modo servidor

```
moon:~# iperf -c alice -i 1
-----
Client connecting to alice, TCP port 5001
TCP window size: 323 KByte (default)
-----
[ 3] local 10.1.0.1 port 33126 connected with 10.1.0.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec    233 MBytes    1.96 Gbits/sec
[ 3] 1.0000-2.0000 sec    233 MBytes    1.96 Gbits/sec
[ 3] 2.0000-3.0000 sec    232 MBytes    1.95 Gbits/sec
[ 3] 3.0000-4.0000 sec    232 MBytes    1.95 Gbits/sec
[ 3] 4.0000-5.0000 sec    235 MBytes    1.97 Gbits/sec
[ 3] 5.0000-6.0000 sec    239 MBytes    2.00 Gbits/sec
[ 3] 6.0000-7.0000 sec    233 MBytes    1.95 Gbits/sec
[ 3] 7.0000-8.0000 sec    234 MBytes    1.96 Gbits/sec
[ 3] 8.0000-9.0000 sec    232 MBytes    1.95 Gbits/sec
[ 3] 9.0000-10.0000 sec   235 MBytes    1.97 Gbits/sec
[ 3] 10.0000-10.0028 sec   640 KBytes    1.86 Gbits/sec
[ 3] 0.0000-10.0028 sec   2.28 GBytes    1.96 Gbits/sec
```

```
alice:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.1.0.10 port 5001 connected with 10.1.0.1 port 33126
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-10.0020 sec   2.28 GBytes    1.96 Gbits/sec
```

Como podemos observar en las imágenes se realizó una transferencia de **2.28 GBytes** con un ancho de banda medio de **1.96 Gbits/sec**. También podemos observar que la conexión se ha mantenido estable, manteniendo valores entre **1.95 y 2.00 Gbits/sec** durante la prueba.

¿Hay alguna diferencia de velocidad?

- **Sentido Alice** → **moon**: 2.57 Gbits/sec.
- **Sentido moon** → **alice**: 1.96 Gbits/sec.

Podemos observar una diferencia de 0.61 Gbit/sec. La diferencia se debe a que **moon** consume más recursos de CPU al generar tráfico (modo cliente) que al recibirlo.

3. IPsec: strongSwan

strongSwan es un software de VPN de código abierto que se usa para crear conexiones seguras (VPN) mediante el protocolo IPsec.

Ajuste del MSS (TCP Maximum Segment Size)

El protocolo IPsec añade una "cabecera" adicional (overhead) a cada paquete para gestionar el cifrado y la integridad. Si un paquete TCP ya tiene el tamaño máximo permitido (MTU de 1500 bytes), al añadirle el overhead de IPsec, el paquete se vuelve demasiado grande y es descartado por los routers.

Para solucionar esto, configuramos una regla en la tabla **mangle** de **iptables**. Esta regla "obliga" a los clientes a anunciar un tamaño de segmento (**MSS**) más pequeño (1360 bytes) durante la negociación de la conexión (paquetes SYN y RST).

Comandos a ejecutar en moon y sun:

Para asegurar la conectividad en ambos sentidos (entrante y saliente), aplicamos estas reglas en ambos gateways:

```
# Ajuste para tráfico IPsec entrante
iptables -t mangle -A FORWARD -m policy --pol ipsec --dir in -p tcp -m tcp --tcp-flags
    SYN,RST SYN -m tcpmss --mss 1361:1536 -j TCPMSS --set-mss 1360

# Ajuste para tráfico IPsec saliente
iptables -t mangle -A FORWARD -m policy --pol ipsec --dir out -p tcp -m tcp --tcp-flags
    SYN,RST SYN -m tcpmss --mss 1361:1536 -j TCPMSS --set-mss 1360
```

Para confirmar que se han aplicado las reglas correctamente podemos usar el comando `iptables -t mangle -L FORWARD -v -n`

- **t mangle** : Indica que quieres ver la tabla de "alteración" de paquetes (donde pusimos el ajuste del MSS).
- **L FORWARD** : Lista únicamente las reglas de la cadena de **reenvío**, que es donde aplicamos el ajuste para los clientes.
- **v (Verbose)**: Muestra los contadores de paquetes y bytes. Si los números son mayores a cero, significa que la regla no solo está escrita, sino que está "atrapando" tráfico real.
- **n (Numeric)**: Muestra las direcciones y puertos en formato numérico (más rápido y evita confusiones con nombres de servicios).

```
sun:~# iptables -t mangle -L FORWARD -v -n
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source               destination          policy match dir in pol ipsec tcp flags:0x06/0x0
2 tcpmss match 1361:1536 TCPMSS set 1360      0.0.0.0/0            0.0.0.0/0            policy match dir out pol ipsec tcp flags:0x06/0x0
02 tcpmss match 1361:1536 TCPMSS set 1360
sun:~#
```

¿Por qué funciona el Ping pero no el TCP?

Si olvidas este paso, notarás un síntoma muy común en redes VPN:

- **El ping funciona**: Los paquetes ICMP suelen ser pequeños y no superan el límite de la MTU incluso con el overhead de IPsec.
- **TCP (SSH, Web) falla**: Las conexiones TCP intentan enviar la mayor cantidad de datos posible por paquete. Al superar el límite físico tras el cifrado, los paquetes se pierden y la conexión "se cuelga".

Limpieza y Mantenimiento

Es obligatorio que, tras cada escenario de prueba, se restaure la configuración original del cortafuegos para evitar conflictos entre reglas de distintos ejercicios. Para ello, se utiliza el comando:

- `moon$ iptables-restore < /etc/iptables.flush`
- `sun$ iptables-restore < /etc/iptables.flush`

3.1 IPsec: escenario ikev2/esp-alg-null

ROADWARRIOR (cliente -gateway)
cliente (iniciador) -> carol
gateway(respondedor) -> moon

Cargamos el escenario con `scripts/load-testconfig ikev2-algs/esp-alg-null`.

Comprueba que no es posible alcanzar alicé desde carol y viceversa:

```
alice:~# ping carol
PING carol.strongswan.org (192.168.0.100) 56(84) bytes of data.
^C
--- carol.strongswan.org ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2045ms

alice:~# █

carol:~# ping alice
PING alice.strongswan.org (10.1.0.10) 56(84) bytes of data.
From uml0.strongswan.org (192.168.0.254) icmp_seq=1 Destination Port Unreachable
From uml0.strongswan.org (192.168.0.254) icmp_seq=2 Destination Port Unreachable
^C
--- alice.strongswan.org ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1002ms

carol:~# █
```

Muestra las tablas de enrutamiento de cada equipo:

```
alice:~# ip route show
default via 10.1.0.1 dev eth0 onlink
10.1.0.0/16 dev eth0 proto kernel scope link src 10.1.0.10
10.8.0.1 via 10.8.0.5 dev tun0
10.8.0.5 dev tun0 proto kernel scope link src 10.8.0.6
alice:~# █

carol:~# ip route show
default via 192.168.0.254 dev eth0 onlink
10.8.0.1 via 10.8.0.9 dev tun0
10.8.0.9 dev tun0 proto kernel scope link src 10.8.0.10
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.100
carol:~# █
```

Muestra las iptables de cada equipo:

```

alice:~# iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source destination
alice:~# █

carol:~# iptables -L -v
Chain INPUT (policy ACCEPT 12 packets, 836 bytes)
 pkts bytes target    prot opt in     out     source destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source destination

Chain OUTPUT (policy ACCEPT 7 packets, 760 bytes)
 pkts bytes target    prot opt in     out     source destination
carol:~# █

```

Inicia IPsec en moon y carol

Para iniciar **IPSec** utilizamos el comando `systemctl start strongswan` en ambos equipos.



Nota:

- **IPsec:** Es un conjunto de protocolos de red que proporciona **seguridad** mediante el cifrado, la autenticación y la integridad de los datos transmitidos por IP.
- **strongSwan:** Es el software de código abierto utilizado en este taller para implementar IPsec en sistemas Linux, encargándose de la negociación de claves y la gestión de los túneles VPN.

Examina con atención el contenido del fichero de configuración `/etc/swanctl/swanctl.conf` de strongSwan en moon y en carol para este escenario.

Moon→

```

moon:~# cat /etc/swanctl/swanctl.conf
connections {

    rw {
left    local_addrs = 192.168.0.1

        local {
            auth = pubkey
            certs = moonCert.pem
            id = moon.strongswan.org
        }
        remote {
            auth = pubkey
        }
        children {
            net {

```

Se crea una conexión llamada **rw**.

Moon escucha en la IP **192.168.0.1**.

La autenticación es por **clave pública (certificados)**.

- Moon usa el certificado **moonCert.pem** y se identifica como `moon.strongswan.org`.

Se define una red protegida (**child SA**) llamada **net**:

- La red interna de moon es **10.1.0.0/16**.
- Se usan algoritmos ESP específicos (`null-sha256-x25519`).

La conexión usa **IKEv2** (`version = 2`).

Algoritmos del túnel:

```

    local_ts = 10.1.0.0/16
    esp_proposals = null-sha256-x25519
    }
    }
    version = 2
    mobike = no
    proposals = aes128-sha256-x25519
  }
}

```

Algoritmos para negociar la VPN:

MOBIKE está desactivado (no se permite cambiar de IP durante la conexión).

Los algoritmos de IKE están fijados a **aes128-sha256-x25519**.

'proposals' es para la fase IKE (control) y 'esp_proposals' para la fase CHILD_SA (datos).
IKE gestiona el canal de control y ESP gestiona el túnel de datos.

En resumen: **moon actúa como gateway VPN IPsec**, autenticado por certificados, ofreciendo acceso seguro a su red interna **10.1.0.0/16** mediante IKEv2.

Carol→

Se define una conexión llamada **home**.

Carol usa la IP local **192.168.0.100** y se conecta al gateway **Moon (192.168.0.1)**.

La autenticación es por **clave pública (certificados)**.

- Carol usa el certificado **carolCert.pem** y se identifica como **carol@strongswan.org**.
- El remoto se identifica como **moon.strongswan.org**.

Se define una **child SA** llamada **home**:

- Carol tiene acceso a la red remota **10.1.0.0/16**.

En resumen: **Carol establece una VPN IPsec con Moon** usando certificados y obtiene acceso seguro a la red **10.1.0.0/16**.

```

carol:~# cat /etc/swanctl/swanctl.conf
connections {

    home {
        local_addrs = 192.168.0.100
        remote_addrs = 192.168.0.1

        local {
            auth = pubkey
            certs = carolCert.pem
            id = carol@strongswan.org
        }
        remote {
            auth = pubkey
            id = moon.strongswan.org
        }
        children {
            home {
                remote_ts = 10.1.0.0/16
                esp_proposals = null-sha256-x25519
            }
        }
        version = 2
        mobike = no
        proposals = aes128-sha256-x25519
    }
}

```

Ambos extremos deben tener la misma configuración en las claves IKE. Si no, la conexión fallará con un error porque los extremos no se pusieron de acuerdo y la negociación IKE aborta.

En el caso de moon:

¿Qué interfaz (dirección IP) de moon utiliza la conexión rw?

La conexión `rw` utiliza la dirección IP 192.168.0.1. Esta IP, definida en el parámetro `local_addr`, identifica la interfaz de red de moon donde se establecerá el túnel VPN.

¿Qué versión de IKE soporta?

Soporta la versión 2 de IKE lo vemos en el parámetro `version = 2`

¿Qué propuesta de protocolo de cifrado, integridad y grupo de intercambio de claves Diffie Hellman admite para la asociación de seguridad IKE?

Para la negociación inicial del túnel (fase de control), moon admite la propuesta definida en `proposals = aes128-sha256-x25519` :

- **Cifrado:** AES con clave de 128 bits (`aes128`).
- **Integridad:** Algoritmo hash SHA-256 (`sha256`).
- **Grupo intercambio D-H:** Intercambio de claves mediante la curva elíptica X25519 (`x25519`).

¿Qué propuesta de protocolo de cifrado, integridad y grupo de intercambio de claves Diffie Hellman admite para la asociación de seguridad ESP hija de la asociación de seguridad IKE?

Para la transferencia real de datos dentro del túnel (definida en el bloque `children`), admite `esp_proposals = null-sha256-x25519` :

- **Cifrado:** `null` . Esto significa que los datos de usuario **no se cifran** (se envían de forma legible para facilitar el análisis con herramientas como Wireshark), aunque el túnel se establezca correctamente.
- **Integridad:** Algoritmo hash SHA-256 (`sha256`) .
- **Grupo Diffie-Hellman:** Intercambio de claves mediante la curva elíptica X25519 (`x25519`).

En el caso de carol:

¿Qué interfaz (dirección IP) local y remota utiliza la conexión home?

Utiliza la IP local **192.168.0.100** y se conecta a la IP remota **192.168.0.1** (moon).

¿Qué versión de IKE soporta?

Soporta la **versión 2** (`version = 2`)

¿Qué propuesta de protocolo de cifrado, integridad y grupo de intercambio de claves Diffie Hellman admite para la asociación de seguridad IKE?

Para la negociación inicial del túnel (fase de control), moon admite la propuesta definida en `proposals = aes128-sha256-x25519` :

- **Cifrado:** AES con clave de 128 bits (`aes128`).
- **Integridad:** Algoritmo hash SHA-256 (`sha256`).
- **Grupo intercambio D-H:** Intercambio de claves mediante la curva elíptica X25519 (`x25519`).

¿Qué propuesta de protocolo de cifrado, integridad y grupo de intercambio de claves Diffie Hellman admite para la asociación de seguridad ESP hija de la asociación de seguridad IKE?

Para la transferencia real de datos dentro del túnel (definida en el bloque `children`), admite

`esp_proposals = null-sha256-x25519`:

- **Cifrado:** `null`. Esto significa que los datos de usuario **no se cifran** (se envían de forma legible para facilitar el análisis con herramientas como Wireshark), aunque el túnel se establezca correctamente.
- **Integridad:** Algoritmo hash SHA-256 (`sha256`).
- **Grupo Diffie-Hellman:** Intercambio de claves mediante la curva elíptica X25519 (`x25519`).

Carga las conexiones:

El comando `expect-connection` sirve para poner en marcha los túneles VPN de forma automática. Su función es cargar la configuración y los certificados necesarios para establecer la conexión entre los equipos (como moon y carol) sin tener que introducir manualmente todos los comandos técnicos de `swanctl`.

- `moon$ expect-connection rw`
- `carol$ expect-connection home`

Muestra las tablas de enrutamiento y las iptables de cada equipo. ¿Ha cambiado algo?

Carol:

```
carol:~# expect-connection home
carol:~# ip route show
default via 192.168.0.254 dev eth0 onlink
10.8.0.1 via 10.8.0.9 dev tun0
10.8.0.9 dev tun0 proto kernel scope link src 10.8.0.10
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.100
```

```
carol:~# iptables -L -v
Chain INPUT (policy ACCEPT 349 packets, 109K bytes)
 pkts bytes target    prot opt in     out     source
 destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source
 destination

Chain OUTPUT (policy ACCEPT 384 packets, 109K bytes)
 pkts bytes target    prot opt in     out     source
 destination
```

Moon:

```
moon:~# ip route show
default via 192.168.0.254 dev eth0 onlink
10.1.0.0/16 dev eth1 proto kernel scope link src 10.1.0.1
10.8.0.0/24 via 10.8.0.2 dev tun0
10.8.0.2 dev tun0 proto kernel scope link src 10.8.0.1
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.1
```

```
moon:~# iptables -L -v
Chain INPUT (policy ACCEPT 77216 packets, 3221M bytes)
 pkts bytes target      prot opt in      out     source
 destination

Chain FORWARD (policy ACCEPT 555 packets, 63561 bytes)
 pkts bytes target      prot opt in      out     source
 destination

Chain OUTPUT (policy ACCEPT 68395 packets, 2456M bytes)
 pkts bytes target      prot opt in      out     source
 destination
```

No, no hay cambios visibles. Esto sucede porque **IPsec no utiliza la tabla de enrutamiento estándar ni reglas de iptables** para redirigir el tráfico. En su lugar, strongSwan instala **políticas de seguridad (XFRM)** directamente en el núcleo de Linux. Estas políticas interceptan automáticamente los paquetes destinados a la red remota (10.1.0.0/16) y los meten en el túnel solo cuando la conexión se ha establecido con éxito.

Levanta ahora la conexión IPsec home desde carol

El comando `swanctl --initiate --child home` realiza las siguientes acciones técnicas:

1. **Inicio de la Negociación (IKEv2):** Carol envía una solicitud a la IP de moon (**192.168.0.1**) para iniciar el intercambio de claves.
2. **Autenticación:** Ambos nodos intercambian y validan sus certificados digitales (`carolCert.pem` y `moonCert.pem`) para confirmar sus identidades.
3. **Creación de la SA Hija (Child SA):** Una vez que se confían el uno en el otro, se crea específicamente el túnel de datos llamado "**home**". Este túnel es el que se encarga de proteger el tráfico hacia la red **10.1.0.0/16**.
4. **Instalación en el Núcleo:** El comando finaliza cuando las políticas de seguridad se instalan en el kernel de Linux, permitiendo que Carol pueda comunicarse con la red de Alice.

Se trata de IPsec con IKEv2 (porque version = 2). Es una VPN site-to-site / road-warrior según la práctica: Carol es un usuario remoto que se conecta a Moon. Usa certificados (pubkey) para autenticarse, no contraseñas ni PSK.

esp: IPsec usa ESP (Encapsulating Security Payload).

null: Sin cifrado

sha256: integridad de datos

x25519: intercambio de claves para establecer el túnel

Modo transporte: solo se cifraría el tráfico entre hosts específicos, no toda la red.

```

carol:~# swanctl --initiate --child home
[IKE] initiating IKE_SA home[1] to 192.168.0.1
[ENC] generating IKE_SA_INIT request 0 [ SA KE No N(NATD_S_IP) N(NATD_D_IP) N(FRAG_SUP) N(HASH_ALG) N(REDIR_SUP) ]
[NET] sending packet: from 192.168.0.100[500] to 192.168.0.1[500] (240 bytes)
[NET] received packet: from 192.168.0.1[500] to 192.168.0.100[500] (273 bytes)
[ENC] parsed IKE_SA_INIT response 0 [ SA KE No N(NATD_S_IP) N(NATD_D_IP) CERTREQ N(FRAG_SUP) N(HASH_ALG) N(CHDLESS_SUP) N(MULT_AUTH) ]
[CFG] selected proposal: IKE:AES_CBC_128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
[IKE] received cert request for "C=CH, O=strongSwan Project, CN=strongSwan Root CA"
[IKE] sending cert request for "C=CH, O=strongSwan Project, CN=strongSwan Root CA"
[IKE] authentication of 'carol@strongswan.org' (myself) with RSA_EM SA_PKCS1_SHA2_256 successful
[IKE] sending end entity cert "C=CH, O=strongSwan Project, OU=Research, CN=carol@strongswan.org"

```

Comprueba que ahora sí es posible la conectividad carol --> alice

Para comprobar que la conectividad entre **carol** y **alice** ya es operativa a través del túnel IPsec, hemos realizado las siguientes pruebas:

- **Ping Carol → Alice:** Sí hay conectividad. El túnel está correctamente establecido y las políticas están desviando el tráfico hacia la red interna de **moon**.
- **SSH Carol → Alice:** Se conecta correctamente.

```

carol:~# ssh 10.1.0.10
Warning: Permanently added '10.1.0.10' (ECDSA) to the list of known hosts.
alice:~# ss -4an
Netid State  Recv-Q Send-Q Local Address:Port      Peer Address:Port
Process
udp    UNCONN 0      0      0.0.0.0:37958        0.0.0.0:*
tcp    LISTEN 0      128     0.0.0.0:22          0.0.0.0:*
tcp    ESTAB  0      0      10.1.0.10:22        192.168.0.100:46974

```

- **Traceroute Carol → Alice:**

```

carol:~# traceroute alice
traceroute to alice (10.1.0.10), 30 hops max, 60 byte packets
 1 moon1.strongswan.org (10.1.0.1)  0.963 ms  0.869 ms  0.930 ms
 2 alice.strongswan.org (10.1.0.10)  8.568 ms  8.454 ms  8.552 ms

```

Muestra las reglas de enrutado mediante `ip rule` y observa que se ha añadido la tabla de enrutamiento que utiliza strongSwan con identificador 220. Muestre dicha tabla mediante `ip route show table 220`.

```
carol:~# ip rule
0:      from all lookup local
220:    from all lookup 220
32766:  from all lookup main
32767:  from all lookup default
carol:~# ip route show table 220
10.1.0.0/16 via 192.168.0.1 dev eth0 proto static src 192.168.0.100
```

La salida del comando `ip rule` muestra que la regla 220 tiene una prioridad mayor que (Número más bajo) que la tabla principal `main` o la `default`.

- **Consecuencia:** Cada vez que Carol envía un paquete, el sistema mira primero la tabla 220. Si el destino coincide con la regla de esa tabla, el paquete sigue esa ruta; si no, pasa a la siguiente tabla.

La salida del comando `ip route show table 220` nos dice explícitamente lo que pasa con el tráfico hacia Alice: `10.1.0.0/16 via 192.168.0.1 dev eth0 proto static src 192.168.0.100`

- `10.1.0.0/16`: Es el destino (la red interna detrás de Moon).
- `via 192.168.0.1`: Indica que el paquete debe enviarse a **Moon**, que actúa como el gateway del túnel.
- `src 192.168.0.100`: Fuerza a que el paquete salga con la dirección IP de Carol, asegurando que Moon sepa quién envía la información.

Comprueba si es posible hacer un ping carol --> moon o moon --> carol.

Si es posible realizar pin entre Carol (192.168.0.100) y Moon (192.168.0.1) en ambas direcciones.

```
carol:~# ping moon
PING moon.strongswan.org (192.168.0.1) 56(84) bytes of data.
64 bytes from moon.strongswan.org (192.168.0.1): icmp_seq=1 ttl=64 ti
me=1.79 ms
64 bytes from moon.strongswan.org (192.168.0.1): icmp_seq=2 ttl=64 ti
me=1.01 ms
^C
--- moon.strongswan.org ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 1.014/1.404/1.794/0.390 ms
```

```
moon:~# ping carol
PING carol.strongswan.org (192.168.0.100) 56(84) bytes of data.
64 bytes from carol.strongswan.org (192.168.0.100): icmp_seq=1 tt
l=64 time=0.841 ms
64 bytes from carol.strongswan.org (192.168.0.100): icmp_seq=2 tt
l=64 time=0.979 ms
^C
--- carol.strongswan.org ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1014ms
rtt min/avg/max/mdev = 0.841/0.910/0.979/0.069 ms
```

Muestra las conexiones IPsec en alice, moon y carol con el comando `swanctl --list-sas`:

Carol:

```

carol:~# swanctl --list-sas
home: #1, ESTABLISHED, IKEv2, 0badbcf0b369c31b_i* 3785d8719bee04e0_r
  local 'carol@strongswan.org' @ 192.168.0.100[500]
  remote 'moon.strongswan.org' @ 192.168.0.1[500]
  AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
  established 7394s ago, rekeying in 6829s
home: #3, reqid 1, INSTALLED, TUNNEL, ESP:NULL/HMAC_SHA2_256_128/CU
RVE_25519
  installed 840s ago, rekeying in 2438s, expires in 3120s
  in cf006686, 2276 bytes, 50 packets, 54s ago
  out cc1f2867, 2338 bytes, 33 packets, 54s ago
  local 192.168.0.100/32
  remote 10.1.0.0/16
No leaks detected, 1464 suppressed by whitelist
carol:~# █

```

Moon:

```

moon:~# swanctl --list-sas
rw: #1, ESTABLISHED, IKEv2, 0badbcf0b369c31b_i 3785d8719bee04e0_r
*
  local 'moon.strongswan.org' @ 192.168.0.1[500]
  remote 'carol@strongswan.org' @ 192.168.0.100[500]
  AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
  established 7515s ago, rekeying in 6756s
net: #3, reqid 1, INSTALLED, TUNNEL, ESP:NULL/HMAC_SHA2_256_128
/CURVE_25519
  installed 961s ago, rekeying in 2302s, expires in 2999s
  in cc1f2867, 2338 bytes, 33 packets, 175s ago
  out cf006686, 2276 bytes, 50 packets, 175s ago
  local 10.1.0.0/16
  remote 192.168.0.100/32
No leaks detected, 1464 suppressed by whitelist
moon:~# █

```

Alice:

```

root@omnetvm2024:/home/user/Downloads# ssh 10.1.0.10
alice:~# swanctl --list-sas
connecting to 'unix:///var/run/charon.vici' failed: No such file or d
irectory
Error: connecting to 'default' URI failed: No such file or directory
StrongSwan 5.9.8 swanctl
usage:
  swanctl --list-sas [--ike <name>|--ike-id <id>] [--child <name>|--c
hild-id <id>]
                        [--raw|--pretty]
  --help                (-h)  show usage information
  --ike                 (-i)  filter IKE_SAs by name
  --ike-id              (-I)  filter IKE_SAs by unique identifie
r
  --child               (-c)  filter CHILD_SAs by name
  --child-id            (-C)  filter CHILD_SAs by unique identif
ier
  --noblock             (-n)  don't wait for IKE_SAs in use
  --raw                 (-r)  dump raw response message
  --pretty              (-P)  dump raw response message in prett
y print
  --debug               (-v)  set debug level, default: 1
  --options              (+)  read command line options from fil

```

El error observado en el host **Alice** al ejecutar `swanctl --list-sas` es el comportamiento esperado. Esto se debe a que Alice actúa como un host final dentro de la red privada y no como un extremo del túnel IPsec. La asociación de seguridad (SA) se gestiona exclusivamente entre **Moon** y **Carol**. Por tanto, Alice no tiene activo el demonio `charon`, lo que impide que la herramienta `swanctl` encuentre el socket de comunicación necesario para listar las conexiones.

¿Cuál es el algoritmo de encriptado usado en la IKE SA? ¿Y en la ESP SA?

- IKE SA (Asociación de Control):
 - Encriptado: `AES_CBC_128`.
- ESP SA (Asociación de Datos/Hija):
 - Encriptado: `NULL`.

¿Cuál es el algoritmo de integridad usado en la IKE SA? ¿Y en la ESP SA?

- IKE SA (Asociación de Control):
 - Integridad: `HMAC_SHA2_256_128`.
- ESP SA (Asociación de Datos/Hija):
 - Integridad: `HMAC_SHA2_256_128`.

¿Cuál es algoritmo usado para el Diffie-Hellman en IKE?

El algoritmo utilizado para el intercambio de claves es `CURVE_25519` (también conocido como X25519 o algoritmo de curva elíptica).

¿Cuánto falta para se haga un rekeying de la IKE SA? ¿Y de la ESP SA?

El "rekeying" es el tiempo que falta para que el sistema genere nuevas claves de seguridad. Como podemos ver en las capturas es:

- **IKE SA:** 6829s en Carol y 6756s en Moon.
- **ESP SA:** 2438s en Carol y 2302s en Moon.

¿Cuáles son los SPI de la IKE SA? ¿Cuál corresponde a carol y cuál a moon?

Se encuentran al final de la primera línea de la conexión:

- **Carol (Iniciador, `_i`):** `Obadbcf0b369c31b`.
- **Moon (Respondedor, `_r`):** `3785d8719bee04e0`.

¿Cuáles son los SPI de la ESP SA? ¿Cuál corresponde a carol y cuál a moon?

Son los códigos hexadecimales que aparecen junto a `in` y `out`. Como podemos observar están invertidos entre los dos equipos por que lo que para uno sale, para el otro entra.

Dirección del Tráfico	SPI en Carol	SPI en Moon
Entrada (<code>in</code>)	<code>cf006686</code>	<code>cc1f2867</code>
Salida (<code>out</code>)	<code>cc1f2867</code>	<code>cf006686</code>

Mide la velocidad alicé --> carol y viceversa con iperf tanto con TCP como con UDP.

Dirección: Alicé → Carol: En esta prueba, **carol** actúa como servidor y **alice** como cliente que envía los datos.

Prueba UDP

- En carol: `iperf -s -u`
- En alicé: `iperf -c 192.168.0.100 -u -b 100M -l 1300` (el parámetro `-b` limita el ancho de banda para evitar colapsar el túnel).

```
UDP buffer size: 208 KByte (default)
.....
[ 3] local 10.1.0.10 port 33526 connected with 192.168.0.100 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-10.0003 sec  93.5 MBytes  78.4 Mbits/sec
[ 3] Sent 75431 datagrams
[ 3] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter  Lost/Tota
[ 3] 0.0000-9.9984 sec  93.5 MBytes  78.5 Mbits/sec  0.090 ms  0/75430 (0%)
alice:~# █

carol:~# iperf -s -u
.....
Server listening on UDP port 5001
UDP buffer size: 208 KByte (default)
.....
[ 3] local 192.168.0.100 port 5001 connected with 10.1.0.10 port 33526
[ ID] Interval      Transfer    Bandwidth      Jitter  Lost/Tota
[ 3] 0.0000-9.9984 sec  93.5 MBytes  78.5 Mbits/sec  0.091 ms  0/75430 (0%)
```

Se ha alcanzado una velocidad de **78.5 Mbits/sec** en Alicé y **78.5 Mbits/sec** en Carol.

Prueba TCP

- En carol: `iperf -s`
- En alicé: `iperf -c 192.168.0.100`

```
alice:~# iperf -c 192.168.0.100
.....
Client connecting to 192.168.0.100, TCP port 5001
TCP window size: 45.0 KByte (default)
.....
[ 3] local 10.1.0.10 port 47938 connected with 192.168.0.100 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-10.0251 sec  176 MBytes  147 Mbits/sec
alice:~# █
```

```
carol:~# iperf -s
.....
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
.....
[ 4] local 192.168.0.100 port 5001 connected with 10.1.0.10 port 47938
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-10.0496 sec  176 MBytes  146 Mbits/sec
carol:~# █
```

Se ha alcanzado una velocidad de **147 Mbits/sec** en alicé y **146 Mbits/sec** en Carol.

Dirección: Carol → Alicé: En esta prueba, **alice** actúa como servidor y **carol** como cliente.

Prueba UDP

- En alicé: `iperf -s -u`
- En carol: `iperf -c 10.1.0.10 -u -b 100M -l 1300`

Prueba TCP

- En alicé: `iperf -s`
- En carol: `iperf -c 10.1.0.10`

```

75430 (0%)
alice:~# iperf -s -u
Server listening on UDP port 5001
UDP buffer size: 208 KByte (default)
-----
[ 3] local 10.1.0.10 port 5001 connected with 192.168.0.100 port 363
44
[ ID] Interval      Transfer    Bandwidth      Jitter  Lost/Tota
l Datagrams
[ 3] 0.0000-9.9984 sec  90.2 MBytes  75.6 Mbits/sec  0.105 ms  0/
72717 (0%)
-----
[ 3] local 192.168.0.100 port 36344 connected with 10.1.0.10 port 50
91
[ ID] Interval      Transfer    Bandwidth      Jitter  Lost/Tota
l Datagrams
[ 3] 0.0000-10.0005 sec  90.2 MBytes  75.6 Mbits/sec
[ 3] Sent 72718 datagrams
[ 3] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter  Lost/Tota
l Datagrams
[ 3] 0.0000-9.9984 sec  90.2 MBytes  75.6 Mbits/sec  0.104 ms  0/
72717 (0%)
carol:~#

```

Se ha alcanzado una velocidad de **75.6 Mb/s** en Alice y **75.6 Mb/s** en Carol.

```

alice:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.1.0.10 port 5001 connected with 192.168.0.100 port 477
40
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-10.0097 sec  84.1 MBytes  70.5 Mbits/sec
-----
carol:~# iperf -c 10.1.0.10
Client connecting to 10.1.0.10, TCP port 5001
TCP window size: 72.0 KByte (default)
-----
[ 3] local 192.168.0.100 port 47740 connected with 10.1.0.10 port 50
91
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-10.0102 sec  84.1 MBytes  70.5 Mbits/sec
carol:~#

```

Se ha alcanzado una velocidad de **78.5 Mb/s** en Alice y **78.5 Mb/s** en Carol.

Paramos el servicio de strongswan:

- `moon$ systemctl stop strongswan`
- `carol$ systemctl stop strongswan`

3.2 IPsec: escenario host2host

Host to host

3.2.1 host2host ESP modo túnel

Cargamos el escenario usando : `omnetvm2024$ scripts/load-testconfig ikev2/host2host-cert`

Primero lo que tenemos que hacer es carga las reglas iptables que eliminan la conectividad entre máquinas excepto mediante IPsec:

- `moon$ iptables-restore < /etc/iptables.rules`
- `sun$ iptables-restore < /etc/iptables.rules`

Ahora comprobamos que no hay conectividad entre ambos haciendo ping `sun → moon` y `moon → sun`:

```

moon:~# ping sun
PING sun.strongswan.org (192.168.0.2) 56(84) bytes of data.
^C
--- sun.strongswan.org ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4087ms
moon:~#

sun:~# ping moon
PING moon.strongswan.org (192.168.0.1) 56(84) bytes of data.
^C
--- moon.strongswan.org ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3062ms
sun:~#

```

En ambos casos no hubo respuesta, confirmando que las reglas bloquean todo el tráfico directo (incluido ICMP), excepto el destinado a establecer túneles IPsec.

Al aplicar el archivo `iptables.rules`, se establece una política de seguridad de tipo 'whitelist' donde todo lo que no está permitido se descarta por defecto. El `ping` falla actualmente porque el protocolo **ICMP** no figura en la lista de excepciones. Sin embargo, las reglas sí permiten el tráfico **IKE (UDP 500)** y **ESP (IP 50)**, lo que permitirá establecer la VPN. Una vez que el túnel esté activo, el tráfico ICMP circulará encapsulado bajo el protocolo ESP y será aceptado por el firewall.

Iniciamos **strongSwan** en moon y sun:

- `moon$ systemctl start strongswan`
- `sun$ systemctl start strongswan`

Activa el demonio (programa en segundo plano) llamado charon, que es el cerebro de IPsec

Contenido de `/etc/swanctl/swanctl.conf` en moon:

```

moon:~# cat /etc/swanctl/swanctl.conf
connections {

    rw {
        local_addrs = 192.168.0.1

        local {
            auth = pubkey
            certs = moonCert.pem
            id = moon.strongswan.org
        }
        remote {
            auth = pubkey
        }
        children {
            net {
                local_ts = 10.1.0.0/16
                esp_proposals = null-sha256-x25519
            }
        }
        version = 2
        mobike = no
        proposals = aes128-sha256-x25519
    }
}

```

Contenido de /etc/swanctl/swanctl.conf en sun:

```

sun:~# cat /etc/swanctl/swanctl.conf
connections {

    gw-gw {
        local_addrs = 192.168.0.2
        remote_addrs = 192.168.0.1

        local {
            auth = pubkey
            certs = sunCert.pem
            id = sun.strongswan.org
        }
        remote {
            auth = pubkey
            id = moon.strongswan.org
        }
        children {
            net-net {
                local_ts = 10.2.0.0/16
            }
        }
    }
}

```

```

remote_ts = 10.1.0.0/16

updown = /usr/local/libexec/ipsec/_updown iptables
rekey_time = 5400
rekey_bytes = 500000000
rekey_packets = 1000000
esp_proposals = aes128gcm128-x25519
}
}
version = 2
mobike = no
reauth_time = 10800
proposals = aes128-sha256-x25519
}
}

```

En el caso de moon:

¿Qué interfaz (dirección IP) de moon utiliza la conexión host-host?

Utiliza la dirección IP 192.168.0.1, definida en el parámetro `local_addr`. Esta es la interfaz a través de la cual Moon se comunicará con la IP de Sun (192.168.0.2)

¿Qué versión de IKE soporta?

La versión 2

¿Qué propuesta de protocolo de cifrado, integridad y grupo de intercambio de claves Diffie Hellman admite para la asociación de seguridad IKE?

Para la fase de control (negociación), moon admite la propuesta `aes128-sha256-x25519`:

- **Cifrado:** AES con clave de 128 bits (`aes128`).
- **Integridad:** Algoritmo hash SHA-256 (`sha256`).
- **Grupo Diffie-Hellman:** Intercambio de claves mediante la curva elíptica X25519 (`x25519`).

¿Qué propuesta de protocolo de cifrado, integridad y grupo de intercambio de claves Diffie Hellman admite para la asociación de seguridad ESP hija de la asociación de seguridad IKE?

Para la transferencia de datos, admite la propuesta `aes128gcm128-x25519`:

- **Cifrado e Integridad:** Utiliza **AES-GCM** con una clave de 128 bits y un ICV de 128 bits (`aes128gcm128`).

Nota técnica: A diferencia del escenario anterior, GCM es un algoritmo de cifrado autenticado (AEAD) que proporciona cifrado y protección de integridad simultáneamente en un solo paso.

- **Grupo Diffie-Hellman:** Se mantiene el uso de la curva elíptica X25519 (`x25519`) para la regeneración de claves (Perfect Forward Secrecy).

Muestra las tablas de enrutamiento y las iptables de cada equipo. ¿Cambió algo?

```
moon:~# ip route show
default via 192.168.0.254 dev eth0 onlink
10.1.0.0/16 dev eth1 proto kernel scope link src 10.1.0.1
10.8.0.0/24 via 10.8.0.2 dev tun0
10.8.0.2 dev tun0 proto kernel scope link src 10.8.0.1
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.1
#
```

```
moon:~# iptables -L -v
Chain INPUT (policy DROP 185 packets, 7770 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0    0 ACCEPT     esp  --  eth0   any     anywhere             anywhere
    0    0 ACCEPT     ah   --  eth0   any     anywhere             anywhere
    0    0 ACCEPT     udp  --  eth0   any     anywhere             anywhere          udp spt:isakmp dpt:isakmp
    0    0 ACCEPT     udp  --  eth0   any     anywhere             anywhere          udp spt:ipsec-nat-t dpt:ipsec-nat-t
  430 36502 ACCEPT     tcp  --  any    any     anywhere             anywhere          tcp dpt:ssh
    0    0 ACCEPT     tcp  --  eth0   any     winnetou.strongswan.org anywhere          tcp spt:http

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy DROP 490 packets, 20580 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0    0 ACCEPT     esp  --  any    eth0    anywhere             anywhere
    0    0 ACCEPT     ah   --  any    eth0    anywhere             anywhere
    0    0 ACCEPT     udp  --  any    eth0    anywhere             anywhere          udp spt:isakmp dpt:isakmp
    0    0 ACCEPT     udp  --  any    eth0    anywhere             anywhere          udp spt:ipsec-nat-t dpt:ipsec-nat-t
  283 35476 ACCEPT     tcp  --  any    any     anywhere             anywhere          tcp spt:ssh
    0    0 ACCEPT     tcp  --  any    eth0    anywhere             winnetou.strongswan.org tcp dpt:http
moon:~#
```

```
sun:~# ip route show
default via 192.168.0.254 dev eth0 onlink
10.2.0.0/16 dev eth1 proto kernel scope link src 10.2.0.1
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.2
```

```
sun:~# iptables -L -v
Chain INPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0    0 ACCEPT     esp  --  eth0   any     anywhere             anywhere
    0    0 ACCEPT     ah   --  eth0   any     anywhere             anywhere
    0    0 ACCEPT     udp  --  eth0   any     anywhere             anywhere          udp spt:isakmp dpt:isakmp
    0    0 ACCEPT     udp  --  eth0   any     anywhere             anywhere          udp spt:ipsec-nat-t dpt:ipsec-nat-t
  294 25490 ACCEPT     tcp  --  any    any     anywhere             anywhere          tcp dpt:ssh
    0    0 ACCEPT     tcp  --  eth0   any     winnetou.strongswan.org anywhere          tcp spt:http

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy DROP 4 packets, 336 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0    0 ACCEPT     esp  --  any    eth0    anywhere             anywhere
    0    0 ACCEPT     ah   --  any    eth0    anywhere             anywhere
    0    0 ACCEPT     udp  --  any    eth0    anywhere             anywhere          udp spt:isakmp dpt:isakmp
    0    0 ACCEPT     udp  --  any    eth0    anywhere             anywhere          udp spt:ipsec-nat-t dpt:ipsec-nat-t
  181 23224 ACCEPT     tcp  --  any    any     anywhere             anywhere          tcp spt:ssh
    0    0 ACCEPT     tcp  --  any    eth0    anywhere             winnetou.strongswan.org tcp dpt:http
sun:~#
```

En las **tables de enrutamiento** no se observa ningún cambio. Sin embargo en **iptables**, tras aplicar `iptables.rules`, la política por defecto ha pasado a ser **DROP**. Actualmente, el cortafuegos solo permite el tráfico necesario para establecer el túnel (IKE y ESP) y la gestión por SSH.

Cargamos las conexiones:

- moon\$ expect-connection host-host
- sun\$ expect-connection host-host

Aunque el programa esté encendido, no sabe qué hacer hasta que lea el archivo de configuración.

Levantamos la conexión host-host en moon:

- moon\$ swanctl --initiate --child host-host

Comprueba que ahora sí es posible la conectividad moon ↔ sun

: Si ves `IKE_SA established` y `CHILD_SA established`, el túnel está arriba.

Antes **no era posible hacer ping**, pero **ahora sí** gracias a que se ha establecido el **túnel IPsec**. Una vez negociadas las claves y activado el túnel, el tráfico ICMP **se encapsula y viaja por el túnel**, permitiendo la **conectividad bidireccional entre Moon y Sun** de forma **segura**.

```
moon:~# ping sun
PING sun.strongswan.org (192.168.0.2) 56(84) bytes of data.
64 bytes from sun.strongswan.org (192.168.0.2): icmp_seq=1 ttl=64 t
time=0.811 ms
64 bytes from sun.strongswan.org (192.168.0.2): icmp_seq=2 ttl=64 t
time=0.850 ms
64 bytes from sun.strongswan.org (192.168.0.2): icmp_seq=3 ttl=64 t
time=0.764 ms
64 bytes from sun.strongswan.org (192.168.0.2): icmp_seq=4 ttl=64 t
time=1.14 ms
^C
--- sun.strongswan.org ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3035ms
rtt min/avg/max/ndev = 0.764/0.891/1.139/0.146 ms

sun:~# ping moon
PING moon.strongswan.org (192.168.0.1) 56(84) bytes of data.
64 bytes from moon.strongswan.org (192.168.0.1): icmp_seq=1 ttl=64
time=0.736 ms
64 bytes from moon.strongswan.org (192.168.0.1): icmp_seq=2 ttl=64
time=1.36 ms
64 bytes from moon.strongswan.org (192.168.0.1): icmp_seq=3 ttl=64
time=0.959 ms
^C
--- moon.strongswan.org ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2043ms
rtt min/avg/max/ndev = 0.736/1.018/1.360/0.258 ms
sun:~#
```

Muestra las reglas de enrutado mediante ip rule y observa que se ha añadido la tabla de enrutamiento que utiliza strongSwan con identificador 220. Muestra dicha tabla mediante ip route show table 220. ¿Qué cambio hizo posible la conectividad moon ↔ sun?

Se ha insertado la regla con prioridad **220**. Esto significa que, para cualquier paquete, el sistema consultará primero la **tabla 220** antes que la tabla **main** (donde están tus rutas de internet normales).

```
moon:~# ip rule
0:      from all lookup local
220:    from all lookup 220
32766:  from all lookup main
32767:  from all lookup default
moon:~#

sun:~# ip rule
0:      from all lookup local
220:    from all lookup 220
32766:  from all lookup main
32767:  from all lookup default
sun:~#

moon:~# ip route show table 220
192.168.0.2 via 192.168.0.2 dev eth0 proto static src 192.168.0.1
moon:~#

sun:~# ip route show table 220
192.168.0.1 via 192.168.0.1 dev eth0 proto static src 192.168.0.2
sun:~#
```

La conectividad fue posible porque se **estableció la CHILD SA (host-host)**, que activó todo lo necesario para que el tráfico pudiera pasar. Esto hizo que el sistema **creara las rutas correctas, activara el cifrado, y permitiera el tráfico en el firewall**. Solo cuando todo esto estuvo activo, los datos pudieron viajar **por el túnel**, de forma **segura**, entre ambos equipos.

Muestra las conexiones IPsec en moon y sun con el comando swanctl -list-sas

```
moon:~# swanctl --list-sas
host-host: #2, ESTABLISHED, IKEv2, 216292cd57b60153_i* 5f251235441
4de2b_r
local 'moon.strongswan.org' @ 192.168.0.1[500]
remote 'sun.strongswan.org' @ 192.168.0.2[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 100s ago, reauth in 9636s
host-host: #3, reqid 1, INSTALLED, TUNNEL, ESP:AES_GCM_16-128
installed 100s ago, rekeying in 5040s, expires in 5040s
in c3757dbe, 0 bytes, 0 packets
out cc38f284, 0 bytes, 0 packets
local 192.168.0.1/32
remote 192.168.0.2/32

sun:~# swanctl --list-sas
host-host: #2, ESTABLISHED, IKEv2, 216292cd57b60153_i 5f2512354414d
e2b_r*
local 'sun.strongswan.org' @ 192.168.0.2[500]
remote 'moon.strongswan.org' @ 192.168.0.1[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 102s ago, reauth in 9986s
host-host: #3, reqid 1, INSTALLED, TUNNEL, ESP:AES_GCM_16-128
installed 102s ago, rekeying in 4968s, expires in 5038s
in cc38f284, 0 bytes, 0 packets
out c3757dbe, 0 bytes, 0 packets
local 192.168.0.2/32
remote 192.168.0.1/32
No leaks detected, 1464 suppressed by whitelist
```

¿Cuál es el algoritmo de encriptado usado en la IKE SA? ¿Y en la ESP SA?

IKE SA (Capa de Control):

- Encriptado: **AES_CBC-128**.

ESP SA (Capa de Datos - CHILD_SA):

- Encriptado: **AES_GCM_16-128**.

¿Cuál es el algoritmo de integridad usado en la IKE SA? ¿Y en la ESP SA?

ESP SA (Capa de Datos - CHILD_SA):

- **Integridad:** `AES_GCM_16-128` (Este algoritmo es AEAD, por lo que gestiona cifrado e integridad simultáneamente).

IKE SA (Capa de Control):

- **Integridad:** `HMAC_SHA2_256_128`.

¿Cuál es algoritmo usado para el Diffie-Hellman en IKE?

El algoritmo usado es el de curva elíptica, `CURVE_25519`.

¿Cuánto falta para se haga un rekeying de la IKE SA? ¿Y de la ESP SA?

IKE SA (Capa de Control):

- **Tiempo para Re-autenticación:** Quedan **10.278 segundos**.

ESP SA (Capa de Datos - CHILD_SA):

- **Tiempo para Rekeying:** Quedan **4.919 segundos**.

¿Cuáles son los SPI de la IKE SA? ¿Cuál corresponde a moon y cuál a sun?

- **Moon:** `9cefc4c648fc4b0` (marcado con `_I` de *initiator*, ya que Moon inició la conexión).
- **Sun:** `3f073e1281f0f48e` (marcado con `_R` de *responder*).

¿Cuáles son los SPI de la ESP SA? ¿Cuál corresponde a moon y cuál a sun?

En IPsec, cada nodo genera el SPI para el tráfico que **recibe**.

- **Sun:** `c18ae45c`
- **Moon:** `c95cf2f2`

Mide la velocidad moon → sun y viceversa con iperf tanto con TCP como con UDP.

Prueba TCP (Ancho de banda máximo)

- **Moon → Sun:**
 - En **Sun** (Servidor): `iperf -s`
 - En **Moon** (Cliente): `iperf -c 192.168.0.2`
- **Sun → Moon:**
 - En **Moon** (Servidor): `iperf -s`
 - En **Sun** (Cliente): `iperf -c 192.168.0.1`

Prueba UDP (Latencia y pérdida)

- **Moon → Sun:**
 - En **Sun** (Servidor): `iperf -s -u`
 - En **Moon** (Cliente): `iperf -c 192.168.0.2 -u -b 100M`
- **Sun → Moon:**
 - En **Sun** (Cliente): `iperf -c 192.168.0.1 -u -b 100M`
 - En **Moon** (Servidor): `iperf -s -u`

3.2.2 host2host AH modo transporte

Modifica el contenido de `/etc/swanctl/swanctl.conf` tanto en moon como en sun para que la SA IPsec sea mediante AH en modo transporte, simplemente cambiando en la conexión host-host la directiva `esp_proposals` por `ah_proposals = aesxcbc` y añadiendo justo a continuación la directiva `mode = transport`.

```

local {
    auth = pubkey
    certs = moonCert.pem
    id = moon.strongswan.org
}
remote {
    auth = pubkey
    id = sun.strongswan.org
}
children {
    host-host {
        updown = /usr/local/libexec/ipsec/_updown iptables
        rekey_time = 5400
        rekey_bytes = 500000000
        rekey_packets = 1000000
        ah_proposals = aesxcbc
        mode = transport
    }
}
version = 2
mobike = no
reauth_time = 10800
proposals = aes128-sha256-x25519
}
moon:~#

id = sun.strongswan.org
}
remote {
    auth = pubkey
    id = moon.strongswan.org
}
children {
    host-host {
        updown = /usr/local/libexec/ipsec/_updown iptables
        rekey_time = 5400
        rekey_bytes = 500000000
        rekey_packets = 1000000
        ah_proposals = aesxcbc
        mode = transport
    }
}
version = 2
mobike = no
reauth_time = 10800
proposals = aes128-sha256-x25519
}
sun:~#

```

- **Protocolo AH (`ah_proposals = aesxcbc`)**: A diferencia de ESP, el protocolo AH **no cifra los datos**. Su función es garantizar la **integridad** y la **autenticación**; es decir, asegura que nadie ha modificado el paquete y que el emisor es quien dice ser, usando el algoritmo **AES-XCBC**.
- **Modo Transporte (`mode = transport`)**: En este modo, **no se crea una nueva cabecera IP**. La cabecera AH se inserta directamente entre la cabecera IP original y los datos (ICMP/TCP). Esto es más eficiente para comunicaciones directas entre dos hosts porque reduce el tamaño del paquete.

Reinicia IPsec en ambos equipos y levanta la conexión host-host.

- `systemctl restart strongswan`
- `expect-connection host-host`
- `swanctl --initiate --child host-host`

Muestra las conexiones IPsec en moon. Comprueba que, efectivamente, la SA IPsec host-host utiliza AH en modo transporte.

```

moon:~# swanctl --list-sas
host-host: #2, ESTABLISHED, IKEv2, 216292cd57b60153_i* 5f2512354414
de2b_r
local 'moon.strongswan.org' @ 192.168.0.1[500]
remote 'sun.strongswan.org' @ 192.168.0.2[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 6494s ago, reauth in 3242s
host-host: #4, reqid 1, INSTALLED, TUNNEL, ESP:AES_GCM_16-128/CUR
VE_25519
  installed 1424s ago, rekeying in 3848s, expires in 4516s
  in c217d60c, 0 bytes, 0 packets
  out cf8a8528, 0 bytes, 0 packets
  local 192.168.0.1/32
  remote 192.168.0.2/32
host-host: #5, reqid 1, INSTALLED, TRANSPORT, AH:AES_XCBC_96
installed 76s ago, rekeying in 5290s, expires in 5864s
  in cf5a288a, 320 bytes, 5 packets, 44s ago
  out cf9884bd, 320 bytes, 5 packets, 44s ago
  local 192.168.0.1/32
  remote 192.168.0.2/32
No leaks detected, 1464 suppressed by whitelist
moon:~#

sun:~# swanctl --list-sas
host-host: #2, ESTABLISHED, IKEv2, 216292cd57b60153_i 5f2512354414d
e2b_r*
local 'sun.strongswan.org' @ 192.168.0.2[500]
remote 'moon.strongswan.org' @ 192.168.0.1[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 6496s ago, reauth in 3592s
host-host: #4, reqid 1, INSTALLED, TUNNEL, ESP:AES_GCM_16-128/CUR
VE_25519
  installed 1426s ago, rekeying in 3742s, expires in 4514s
  in cf8a8528, 0 bytes, 0 packets
  out c217d60c, 0 bytes, 0 packets
  local 192.168.0.2/32
  remote 192.168.0.1/32
host-host: #5, reqid 1, INSTALLED, TRANSPORT, AH:AES_XCBC_96
installed 78s ago, rekeying in 4824s, expires in 5862s
  in cf9884bd, 320 bytes, 5 packets, 47s ago
  out cf5a288a, 320 bytes, 5 packets, 47s ago
  local 192.168.0.2/32
  remote 192.168.0.1/32
No leaks detected, 1464 suppressed by whitelist
sun:~#

```

Como podemos ver la salida del comando `swanctl --list-sas` confirma que se ha migrado correctamente a la configuración **AH en modo transporte**. Lo podemos ver claramente en esta línea: `host-host: #5, reqid 1, INSTALLED, TRANSPORT, AH: AES_XCBC_96`:

- **TRANSPORT**: Confirma que se ha abandonado el modo túnel. Ahora no se encapsula el paquete en una nueva cabecera IP, sino que se inserta la seguridad directamente en el paquete original entre los dos hosts.
- **AH:AES_XCBC_96**: Este es el cambio más importante. Confirma que se está usando el protocolo **AH** con el algoritmo **AES-XCBC** para garantizar que nadie modifique los datos.

3.2.3 host2host ESP modo transporte

Para poner **ESP en modo transporte** tenemos que realizar las siguientes modificaciones en el archivo `/etc/swanctl/swanctl.conf` tanto en **Moon** como en **Sun**:

- Cambiamos `ah_proposals` por `esp_proposals = aes128gcm128`
- Mantenemos `mode = transport`

```
local {
    auth = pubkey
    certs = moonCert.pem
    id = moon.strongswan.org
}
remote {
    auth = pubkey
    id = sun.strongswan.org
}
children {
    host-host {
        updown = /usr/local/libexec/ipsec/_updown iptables
        rekey_time = 5400
        rekey_bytes = 500000000
        rekey_packets = 1000000
        esp_proposals = aes128gcm128
        mode = transport
    }
}
version = 2
mobike = no
reauth_time = 10800
proposals = aes128-sha256-x25519
}
moon:~#
```

```
id = sun.strongswan.org
}
remote {
    auth = pubkey
    id = moon.strongswan.org
}
children {
    host-host {
        updown = /usr/local/libexec/ipsec/_updown iptables
        rekey_time = 5400
        rekey_bytes = 500000000
        rekey_packets = 1000000
        esp_proposals = aes128gcm128
        mode = transport
    }
}
version = 2
mobike = no
reauth_time = 10800
proposals = aes128-sha256-x25519
}
sun:~#
```

Reiniciamos el IPsec en ambos equipos con:

- `systemctl restart strongswan`
- `expect-connection host-host`
- `swanctl --initiate --child host-host`

Y ahora con la salida del comando `swanctl --list-sas` podemos ver que se ha cambiado correctamente al protocolo **ESP en modo transporte**. Lo podemos confirmar en la línea `host-host: #1, reqid 1, INSTALLED, TRANSPORT, ESP: AES_GCM_16-128`

```
moon:~# swanctl --list-sas
host-host: #1, ESTABLISHED, IKEv2, 6682e172b1118046_i* fd784f737f981
1a5e_r
local 'moon.strongswan.org' @ 192.168.0.1[500]
remote 'sun.strongswan.org' @ 192.168.0.2[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 12s ago, reauth in 10245s
host-host: #1, reqid 1, INSTALLED, TRANSPORT, ESP:AES_GCM_16-128
installed 13s ago, rekeying in 4867s, expires in 5928s
in cad730c3, 0 bytes, 0 packets
out c133594d, 0 bytes, 0 packets
local 192.168.0.1/32
remote 192.168.0.2/32
No leaks detected, 1464 suppressed by whitelist
moon:~#
```

```
sun:~# swanctl --list-sas
host-host: #1, ESTABLISHED, IKEv2, 6682e172b1118046_i fd784f737f981
a5e_r*
local 'sun.strongswan.org' @ 192.168.0.2[500]
remote 'moon.strongswan.org' @ 192.168.0.1[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 17s ago, reauth in 10560s
host-host: #1, reqid 1, INSTALLED, TRANSPORT, ESP:AES_GCM_16-128
installed 17s ago, rekeying in 4870s, expires in 5923s
in c133594d, 0 bytes, 0 packets
out cad730c3, 0 bytes, 0 packets
local 192.168.0.2/32
remote 192.168.0.1/32
No leaks detected, 1464 suppressed by whitelist
sun:~#
```

3.2.4 host2host AH modo túnel

Para poner **AH en modo túnel** tenemos que realizar las siguientes modificaciones en el archivo `/etc/swanctl/swanctl.conf` tanto en **Moon** como en **Sun**:

- Cambiamos otra vez `esp_proposals` por `ah_proposals = aesxcbc`
- Cambiamos de `mode= transport` por `mode=tunnel`

Reiniciamos el IPsec en ambos equipos con:

- `systemctl restart strongswan`
- `expect-connection host-host`

- `swanctl --initiate --child host-host`

Y ahora con el la salida del comando `swanctl --list-sas` podemos ver que se ha cambiado correctamente al protocolo **AH en modo túnel** . Lo podemos ver en la línea `host-host: #1, reqid 1, INSTALLED, TUNNEL, AH: AES_XCBC_96`

```
moon:~# swanctl --list-sas
host-host: #1, ESTABLISHED, IKEv2, d1d6b0aad0e881c7_i* 0e9e74d6b2114055_r
local 'moon.strongswan.org' @ 192.168.0.1[500]
remote 'sun.strongswan.org' @ 192.168.0.2[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 12s ago, reauth in 10605s
host-host: #1, reqid 1, INSTALLED, TUNNEL, AH:AES_XCBC_96
installed 12s ago, rekeying in 5300s, expires in 5928s
in c079719a, 0 bytes, 0 packets
out cf4ca861, 0 bytes, 0 packets
local 192.168.0.1/32
remote 192.168.0.2/32
No leaks detected, 1464 suppressed by whitelist
moon:~#
```

```
sun:~# swanctl --list-sas
host-host: #1, ESTABLISHED, IKEv2, d1d6b0aad0e881c7_i 0e9e74d6b2114055_r*
local 'sun.strongswan.org' @ 192.168.0.2[500]
remote 'moon.strongswan.org' @ 192.168.0.1[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 15s ago, reauth in 10444s
host-host: #1, reqid 1, INSTALLED, TUNNEL, AH:AES_XCBC_96
installed 15s ago, rekeying in 4849s, expires in 5925s
in cf4ca861, 0 bytes, 0 packets
out c079719a, 0 bytes, 0 packets
local 192.168.0.2/32
remote 192.168.0.1/32
No leaks detected, 1464 suppressed by whitelist
sun:~#
```

3.3 IPsec: escenario net2net

3.3.1 net2net modo transporte

El modo transporte no tiene sentido en un escenario **net2net** (red a red) por las siguientes razones técnicas fundamentales:

- **Falta de encapsulación:** El modo transporte no añade una nueva cabecera IP, sino que utiliza la original.
- **Incompatibilidad con IPs privadas:** En una VPN net2net, los paquetes de las subredes internas tienen direcciones privadas (como 10.1.0.x). Al no haber una nueva cabecera que los "envuelva" con las IPs públicas de los gateways, estos paquetes no pueden viajar a través de la red intermedia (Internet).
- **Diseño para Host-to-Host:** El modo transporte está diseñado exclusivamente para la comunicación directa entre los dos puntos finales de la conexión IPsec.
- **Imposibilidad de enrutamiento:** En net2net, los Gateways necesitan transportar tráfico de terceras máquinas (las de sus respectivas redes locales). Sin el **modo túnel**, el Gateway no puede actuar como un puente para esas redes.



De cara al examen:

- Transporte: protege solo los datos del paquete IP → solo útil en host↔host.
- Túnel: Encapsula todo el paquete → necesario en net↔net o gateway↔gateway.
- En net2net, el transporte rompería la comunicación porque las direcciones IP internas no serían enrutables.

Conclusión: El modo transporte no tiene sentido en escenarios net2net, ya que los gateways deben encapsular completamente los paquetes de las redes internas usando el modo túnel para asegurar la correcta entrega y confidencialidad.

3.3.2 net2net ESP modo túnel

Preparación del Escenario

- **Carga de la configuración en el host:**

```
omnetvm2024$ scripts/load-testconfig ikev2/net2net-cert
```

- **Restauración de reglas de Firewall:** Ejecutamos esto en **Moon** y **Sun** para asegurar que solo el tráfico IPsec pueda circular:

```
moon$ iptables-restore < /etc/iptables.rules
sun$ iptables-restore < /etc/iptables.rules
```

Una vez cargada la configuración, arrancamos el servicio y levantamos la conexión net-net desde uno de ellos.

- **En Moon y Sun:** `swanctl --load-all`
- **En Moon:** `swanctl --initiate --child net-net`

Comprueba la conectividad entre las redes 10.1.0.0/16 y 10.2.0.0/16.

Para probar la conectividad entre ambas redes vamos a realizar un ping de Alice ↔ Bob

<pre>alice:~# ping bob PING bob.strongswan.org (10.2.0.10) 56(84) bytes of data. 64 bytes from bob.strongswan.org (10.2.0.10): icmp_seq=1 ttl=62 time=3.59 ms 64 bytes from bob.strongswan.org (10.2.0.10): icmp_seq=2 ttl=62 time=1.96 ms 64 bytes from bob.strongswan.org (10.2.0.10): icmp_seq=3 ttl=62 time=1.83 ms ^C --- bob.strongswan.org ping statistics --- 3 packets transmitted, 3 received, 0% packet loss, time 2011ms rtt min/avg/max/mdev = 1.826/2.457/3.585/0.799 ms alice:~#</pre>	<pre>bob:~# ping alice PING alice.strongswan.org (10.1.0.10) 56(84) bytes of data. 64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=1 ttl=62 time=1.68 ms 64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=2 ttl=62 time=1.83 ms 64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=3 ttl=62 time=1.81 ms ^C --- alice.strongswan.org ping statistics --- 3 packets transmitted, 3 received, 0% packet loss, time 2029ms rtt min/avg/max/mdev = 1.679/1.771/1.827/0.065 ms bob:~#</pre>
---	---

Comprueba las características de la SA IPsec net-net consultando su estado.

Para comprobar las características de la SA IPsec net-net utilizamos el comando `swanctl --list-sas`, cuya salida es la siguiente:

<pre>moon:~# swanctl --list-sas gw-gw: #1, ESTABLISHED, IKEv2, ea48f0988166f982_i* cdb376c47bca9d65 └─ local 'moon.strongswan.org' @ 192.168.0.1[500] remote 'sun.strongswan.org' @ 192.168.0.2[500] AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519 established 2944s ago, reauth in 7000s net-net: #1, reqid 1, INSTALLED, TUNNEL, ESP:AES_GCM_16-128 installed 2944s ago, rekeying in 1959s, expires in 2996s in cd778931, 924 bytes, 11 packets, 64s ago out c41450a3, 924 bytes, 11 packets, 64s ago local 10.1.0.0/16 remote 10.2.0.0/16 No leaks detected, 51 suppressed by whitelist moon:~#</pre>	<pre>sun:~# swanctl --list-sas gw-gw: #1, ESTABLISHED, IKEv2, ea48f0988166f982_i cdb376c47bc a9d65_r* └─ local 'sun.strongswan.org' @ 192.168.0.2[500] remote 'moon.strongswan.org' @ 192.168.0.1[500] AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519 established 2943s ago, reauth in 7517s net-net: #1, reqid 1, INSTALLED, TUNNEL, ESP:AES_GCM_16-128 installed 2943s ago, rekeying in 2198s, expires in 2997s in c41450a3, 924 bytes, 11 packets, 62s ago out cd778931, 924 bytes, 11 packets, 62s ago local 10.2.0.0/16 remote 10.1.0.0/16 No leaks detected, 51 suppressed by whitelist sun:~#</pre>
--	---

3.3.3 net2net AH modo túnel

Para poner Ah en modo túnel en el escenario net2net debemos modificar el archivo

`/etc/swanctl/swanctl.conf`, de forma que la **CHILD_SA** net-net quede tal que así:

<pre>GNU nano 5.4 /etc/swanctl/swanctl.conf auth = pubkey id = sun.strongswan.org } children { net-net { local_ts = 10.1.0.0/16 remote_ts = 10.2.0.0/16 updown = /usr/local/libexec/ipsec/_updown iptables rekey_time = 5400 rekey_bytes = 500000000 rekey_packets = 1000000 ah_proposals = aesxcbc mode = tunnel } } version = 2 mobike = no reauth_time = 10800</pre>	<pre>GNU nano 5.4 /etc/swanctl/swanctl.conf * } children { net-net { local_ts = 10.2.0.0/16 remote_ts = 10.1.0.0/16 updown = /usr/local/libexec/ipsec/_updown iptables rekey_time = 5400 rekey_bytes = 500000000 rekey_packets = 1000000 ah_proposals = aesxcbc } } version = 2 mobike = no reauth_time = 10800 proposals = aes128-sha256-x25519</pre>
--	---

Ejecutamos los siguientes comandos para poner en marcha el túnel:

- En Moon y Sun : `systemctl restart strongswan`
- En Moon y sun : `iptables-restore < /etc/iptables.rules`
- En Moon: `swanctl --initiate --child net`

Para probar que la conexión se ha establecido correctamente usamos el comando `swanctl --list-sas`

```
moon:~# swanctl --list-sas
gw-gw: #3, ESTABLISHED, IKEv2, df4bb7033a0603ed_i* b70bd62ddf144045_r*
local 'moon.strongswan.org' @ 192.168.0.1[500]
remote 'sun.strongswan.org' @ 192.168.0.2[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 15s ago, reauth in 10091s
net-net: #1, reqid 1, INSTALLED, TUNNEL, AH:AES_XCBC_96
installed 15s ago, rekeying in 5042s, expires in 5925s
in c6a0bc8f, 0 bytes, 0 packets
out c6a0bc8f, 0 bytes, 0 packets
local 10.1.0.0/16
remote 10.2.0.0/16
No leaks detected, 51 suppressed by whitelist
moon:~#

sun:~# swanctl --list-sas
gw-gw: #3, ESTABLISHED, IKEv2, df4bb7033a0603ed_i b70bd62ddf144045_r*
local 'sun.strongswan.org' @ 192.168.0.2[500]
remote 'moon.strongswan.org' @ 192.168.0.1[500]
AES_CBC-128/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/CURVE_25519
established 18s ago, reauth in 10552s
net-net: #1, reqid 1, INSTALLED, TUNNEL, AH:AES_XCBC_96
installed 18s ago, rekeying in 4961s, expires in 5922s
in c6a0bc8f, 0 bytes, 0 packets
out c6a0bc8f, 0 bytes, 0 packets
local 10.2.0.0/16
remote 10.1.0.0/16
No leaks detected, 51 suppressed by whitelist
sun:~#
```

También podemos comprobar que tenemos conexión entre ambos realizando un ping Alice ↔ Bob

```
alice:~# ping bob
PING bob.strongswan.org (10.2.0.10) 56(84) bytes of data:
64 bytes from bob.strongswan.org (10.2.0.10): icmp_seq=1 ttl=62 time=2.65 ms
64 bytes from bob.strongswan.org (10.2.0.10): icmp_seq=2 ttl=62 time=1.71 ms
64 bytes from bob.strongswan.org (10.2.0.10): icmp_seq=3 ttl=62 time=1.81 ms
64 bytes from bob.strongswan.org (10.2.0.10): icmp_seq=4 ttl=62 time=6.69 ms
64 bytes from bob.strongswan.org (10.2.0.10): icmp_seq=5 ttl=62 time=2.32 ms
^C
--- bob.strongswan.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4037ms
rtt min/avg/max/mdev = 1.713/3.034/6.687/1.858 ms
alice:~#

bob:~# ping alice
PING alice.strongswan.org (10.1.0.10) 56(84) bytes of data:
64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=1 ttl=62 time=2.22 ms
64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=2 ttl=62 time=1.69 ms
64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=3 ttl=62 time=2.09 ms
64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=4 ttl=62 time=2.03 ms
64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=5 ttl=62 time=2.26 ms
^C
--- alice.strongswan.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4038ms
rtt min/avg/max/mdev = 1.692/2.058/2.259/0.201 ms
bob:~#
```

4. VPN TLS: OpenVPN

Como OpenVPN e IPsec pueden generar conflictos en el manejo de rutas y políticas del kernel (XFRM), lo primero es asegurarnos de que el servicio anterior esté totalmente detenido en **Moon**, **Alice** y **Carol**:

```
host$ systemctl stop strongswan
host$ systemctl disable strongswan
```

Dado que las máquinas virtuales no tienen salida a Internet, seguiremos el procedimiento de instalación manual mediante paquetes `.deb`. Debemos tener los siguientes archivos en una carpeta de la máquina anfitriona (Los descargamos de [Debian -- Paquetes](#)):.

- `openvpn`
- `easy-rsa` (herramienta para la PKI)
- `liblzo2-2` (librería de compresión)
- `libpkcs11-helper1` (ayudante para tokens criptográficos)

Una vez descargados los paquetes se distribuyen a los equipos mediante el comando `scp`.

```
# Comandos de transferencia desde omnet6-vm
omnet6-vm# scp -O *.deb root@10.1.0.1: # Nodo Moon
```

```
omnet6-vm# scp -O *.deb root@10.1.0.10: # Nodo Alice
omnet6-vm# scp -O *.deb root@192.168.0.100: # Nodo Carol
```

Una vez copiados, entramos en cada máquina y ejecutamos la instalación por lotes:

```
moon/alice/carol$ dpkg -i *.deb
```

4.1.2 Creación de la PKI

En este escenario, cada equipo manejará tres elementos fundamentales:

1. **Certificado del host (`.crt`)**: Contiene la clave pública firmada por la CA. Es público y se intercambia durante la conexión.
2. **Clave privada (`.key`)**: Reside únicamente en el equipo y debe mantenerse secreta.
3. **Certificado de la CA (`ca.crt`)**: Clave pública de la CA para verificar la autenticidad de los otros certificados. El certificado de la clave pública de la CA se almacena en todos los equipos de la VPN.

Importante: La clave privada de la CA (`ca.key`) solo debe existir en moon, ya que es el único equipo con autoridad para firmar certificados.

Ejecutamos los siguientes comandos para preparar el directorio de trabajo:

```
# Copiar scripts para evitar que se pierdan en actualizaciones
moon$ cp -a /usr/share/easy-rsa /etc/openvpn/
moon$ cd /etc/openvpn/easy-rsa
```

Ahora vamos a editar el fichero `vars` de `etc/openvpn/easy-rsa` y definimos las variables de identidad:

```
set_var EASYRSA_REQ_COUNTRY "ES"
set_var EASYRSA_REQ_PROVINCE "LCG"
set_var EASYRSA_REQ_CITY "A Coruna"
set_var EASYRSA_REQ_ORG "UDC"
set_var EASYRSA_REQ_EMAIL "diego.lgomez@udc.es"
set_var EASYRSA_REQ_OU "DR"
```

Ejecutamos la secuencia de generación.

1. Inicializar y crear la CA:

```
moon$ ./easyrsa init-pki
moon$ ./easyrsa build-ca nopass
# CN: DR-CA
```

2. Certificado del Servidor (Moon):

```
moon$ ./easyrsa build-server-full moon nopass
```

3. Certificados de los Clientes (Alice y Carol):

```
moon$ ./easyrsa build-client-full alice nopass
moon$ ./easyrsa build-client-full carol nopass
```

4. Parámetros Diffie-Hellman:

```
moon$ ./easyrsa gen-dh
```

Una vez generados, debemos mover los ficheros desde el directorio de la PKI a la ubicación donde OpenVPN los buscará.

En Moon (Servidor)

```
moon$ cd /etc/openvpn/easy-rsa/pki
moon$ cp ca.crt private/moon.key issued/moon.crt dh.pem /etc/openvpn
```

Hacia los Clientes (Alice y Carol)

```
# Para Alice
moon# scp ca.crt private/alice.key issued/alice.crt alice:/etc/openvpn
# Para Carol
moon# scp ca.crt private/carol.key issued/carol.crt carol:/etc/openvpn
```

4.1.3 Configuración del servidor y de los clientes de la VPN

Primero debemos crear el fichero `/etc/openvpn/server.conf` en **moon** con el siguiente contenido, el cual define la subred de la VPN como `10.8.0.0/24` :

```
port 1194
proto udp
dev tun
ca ca.crt
cert moon.crt
key moon.key
dh dh.pem
server 10.8.0.0 255.255.255.0
cipher AES-128-CBC
comp-lzo yes
push "comp-lzo yes"
ifconfig-pool-persist ipp.txt
persist-key
persist-tun
verb 3
```

- **port 1194:** Puerto en el que escucha el servidor VPN.
- **proto udp:** Usa el protocolo UDP.
- **dev tun:** Crea una interfaz de túnel IP.
- **ca / cert / key / dh:** Certificados y claves para autenticación y cifrado.
- **server 10.8.0.0 255.255.255.0:** Red privada asignada a los clientes VPN.
- **cipher AES-128-CBC:** Algoritmo de cifrado usado.
- **comp-lzo / push:** Activa compresión y la envía a los clientes.
- **persist-key / persist-tun:** Mantiene claves y túnel activos.
- **verb 3:** Nivel de detalle de los logs.

En Alice (Cliente Interno)

Creamos el fichero `/etc/openvpn/client.conf` . Alice se conecta a moon a través de su interfaz interna:

- **client:** Indica que es un cliente VPN.
- **proto udp:** Usa el protocolo UDP.
- **dev tun:** Usa un túnel IP.
- **remote 10.1.0.1 1194:** Dirección y puerto del servidor VPN.
- **nobind:** No fija un puerto local.
- **ca / cert / key:** Certificados y claves del cliente.
- **remote-cert-tls server:** Verifica que el remoto sea un servidor válido.
- **cipher AES-128-CBC:** Cifrado usado.
- **comp-lzo no:** Desactiva compresión.

```
client
proto udp
dev tun
remote 10.1.0.1 1194
nobind
ca ca.crt
cert alice.crt
key alice.key
remote-cert-tls server
cipher AES-128-CBC
comp-lzo no
persist-key
persist-tun
verb 3
```

El cliente usa remote!!!

- **persist-key / persist-tun:** Mantiene claves y túnel activos.
- **verb 3:** Nivel medio de logs.

En Carol (Cliente Externo)

Creamos el fichero `/etc/openvpn/client.conf`. A diferencia de Alice, Carol debe apuntar a la dirección IP externa de moon (`192.168.0.1`):

- **client:** Indica que el equipo actúa como cliente VPN.
- **proto udp:** La conexión usa UDP.
- **dev tun:** Se crea un túnel IP virtual.
- **remote 192.168.0.1 1194:** Dirección y puerto del servidor VPN.
- **nobind:** No fija un puerto local específico.
- **ca / cert / key:** Certificados y claves del cliente Carol.
- **remote-cert-tls server:** Verifica que el remoto sea un servidor válido.
- **cipher AES-128-CBC:** Algoritmo de cifrado utilizado.

```
client
proto udp
dev tun
remote 192.168.0.1 1194
nobind
ca ca.crt
cert carol.crt
key carol.key
remote-cert-tls server
cipher AES-128-CBC
comp-lzo no
persist-key
persist-tun
verb 3
```

- **comp-lzo no:** Compresión desactivada.
- **persist-key / persist-tun:** Mantiene la sesión y el túnel activos.
- **verb 3:** Nivel medio de detalle en los registros.

Puesta en marcha y Verificación

1. **Reiniciar:** Ejecuta `reboot` en **moon**, **alice** y **carol** para asegurar que los servicios arranquen con la nueva configuración.

2. **Comprobar logs:** Tras el reinicio, verifica si el servicio ha levantado correctamente buscando entradas de OpenVPN en el log del sistema:

- `cat /var/log/daemon.log | grep ovpn`

```
Dec 24 18:39:02 alice ovpn-client[2098]: net_iface_mtu_set: mtu 1500 for tun0
Dec 24 18:39:02 alice ovpn-client[2098]: net_iface_up: set tun0 up
Dec 24 18:39:02 alice ovpn-client[2098]: net_addr_ptp_v4_add: 10.8.0.6 peer 10.8.0.5 dev tun0
Dec 24 18:39:02 alice ovpn-client[2098]: net_route_v4_add: 10.8.0.1/32 via 10.8.0.5 dev [NULL] table 0 metric -1
Dec 24 18:39:02 alice ovpn-client[2098]: WARNING: this configuration may cache passwords in memory -- use the auth-nocache option to prevent this
Dec 24 18:39:02 alice ovpn-client[2098]: Initialization Sequence Completed
alice:~# █

Dec 24 18:38:59 carol ovpn-client[2062]: net_iface_mtu_set: mtu 1500 for tun0
Dec 24 18:38:59 carol ovpn-client[2062]: net_iface_up: set tun0 up
Dec 24 18:38:59 carol ovpn-client[2062]: net_addr_ptp_v4_add: 10.8.0.10 peer 10.8.0.9 dev tun0
Dec 24 18:38:59 carol ovpn-client[2062]: net_route_v4_add: 10.8.0.1/32 via 10.8.0.9 dev [NULL] table 0 metric -1
Dec 24 18:38:59 carol ovpn-client[2062]: WARNING: this configuration may cache passwords in memory -- use the auth-nocache option to prevent this
Dec 24 18:38:59 carol ovpn-client[2062]: Initialization Sequence Completed
carol:~# █

Dec 24 18:39:02 moon ovpn-server[2195]: alice/10.1.0.10:50622 MULTI: primary virtual IP for alice/10.1.0.10:50622: 10.8.0.6
Dec 24 18:39:02 moon ovpn-server[2195]: alice/10.1.0.10:50622 Data Channel: using negotiated cipher 'AES-256-GCM'
Dec 24 18:39:02 moon ovpn-server[2195]: alice/10.1.0.10:50622 Outgoing Data Channel: Cipher 'AES-256-GCM' initialized with 256 bit key
Dec 24 18:39:02 moon ovpn-server[2195]: alice/10.1.0.10:50622 Incoming Data Channel: Cipher 'AES-256-GCM' initialized with 256 bit key
Dec 24 18:39:02 moon ovpn-server[2195]: alice/10.1.0.10:50622 SENT CONTROL [alice]: 'PUSH_REPLY,comp-lzo yes,route 10.8.0.1,topology net30,ifconfig 10.8.0.6 10.8.0.5,peer-id 1,cipher AES-256-GCM' (status=1)
moon:~# █

root@omnetvm2024:/home/user/Downloads# █
```

3. **Prueba de conectividad:** Verifica que puedes hacer ping a través del túnel (red `10.8.0.x`):

- **Ping a Moon desde Alice y Carol**

```
alice:~# ping 10.8.0.1
PING 10.8.0.1 (10.8.0.1) 56(84) bytes of data.
64 bytes from 10.8.0.1: icmp_seq=1 ttl=64 time=1.17 ms
64 bytes from 10.8.0.1: icmp_seq=2 ttl=64 time=0.597 ms
64 bytes from 10.8.0.1: icmp_seq=3 ttl=64 time=0.640 ms
^C
--- 10.8.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2034ms
rtt min/avg/max/mdev = 0.597/0.800/1.165/0.258 ms
alice:~# █
```

```
carol:~# ping 10.8.0.1
PING 10.8.0.1 (10.8.0.1) 56(84) bytes of data.
64 bytes from 10.8.0.1: icmp_seq=1 ttl=64 time=0.980 ms
64 bytes from 10.8.0.1: icmp_seq=2 ttl=64 time=1.06 ms
64 bytes from 10.8.0.1: icmp_seq=3 ttl=64 time=0.857 ms
^C
--- 10.8.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2021ms
rtt min/avg/max/mdev = 0.857/0.964/1.055/0.081 ms
carol:~# █
```

- **Ping a Alice desde Moon**

```
moon:~# ping alice
PING alice.strongswan.org (10.1.0.10) 56(84) bytes of data.
64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=1 ttl=64 t
ime=0.678 ms
64 bytes from alice.strongswan.org (10.1.0.10): icmp_seq=2 ttl=64 t
ime=0.896 ms
^C
--- alice.strongswan.org ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1020ms
rtt min/avg/max/mdev = 0.678/0.787/0.896/0.109 ms
moon:~#
```

- **Ping a Carol desde Moon**

```
moon:~# ping carol
PING carol.strongswan.org (192.168.0.100) 56(84) bytes of data.
64 bytes from carol.strongswan.org (192.168.0.100): icmp_seq=1 ttl=
64 time=0.507 ms
64 bytes from carol.strongswan.org (192.168.0.100): icmp_seq=2 ttl=
64 time=0.806 ms
^C
--- carol.strongswan.org ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1017ms
rtt min/avg/max/mdev = 0.507/0.656/0.806/0.149 ms
moon:~# s
```

Modificación para conectividad Cliente-a-Cliente

Por defecto, OpenVPN actúa como una estrella donde los clientes solo ven al servidor. Para permitir que **Alice y Carol se vean entre sí** a través de la VPN, debemos añadir una directiva específica en el servidor:

1. Editamos `/etc/openvpn/server.conf` en **moon**.
2. Añademos la siguiente línea al final del fichero:
 - `client-to-client` El proceso OpenVPN gestiona el enrutamiento internamente en el espacio de usuario, sin que el kernel vea pasar los paquetes por la cadena FORWARD.
3. Reiniciamos el servicio en moon: `systemctl restart openvpn@server`.

Para comprobar que la configuración es correcta vamos a hacer un **ping Alice ↔ Carol**. Los equipos tienen las siguientes direcciones:

```
alice:~# ip addr show tun0
13: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc
pfifo_fast state UNKNOWN group default qlen 500
    link/none
    inet 10.8.0.6 peer 10.8.0.5/32 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::2cfd:2a33:f76b:d93/64 scope link stable-privacy
        valid_lft forever preferred_lft forever
alice:~#
```

```
carol:~# ip addr show tun0
12: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc
pfifo_fast state UNKNOWN group default qlen 500
    link/none
    inet 10.8.0.10 peer 10.8.0.9/32 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::71dd:7bdc:add0:cf15/64 scope link stable-privacy
        valid_lft forever preferred_lft forever
carol:~#
```

```
alice:~# ping 10.8.0.10
PING 10.8.0.10 (10.8.0.10) 56(84) bytes of data.
64 bytes from 10.8.0.10: icmp_seq=1 ttl=64 time=1.43 ms
64 bytes from 10.8.0.10: icmp_seq=2 ttl=64 time=2.06 ms
64 bytes from 10.8.0.10: icmp_seq=3 ttl=64 time=1.46 ms
^C
--- 10.8.0.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 1.429/1.648/2.061/0.291 ms
alice:~#
```

```
carol:~# ping 10.8.0.6
PING 10.8.0.6 (10.8.0.6) 56(84) bytes of data.
64 bytes from 10.8.0.6: icmp_seq=1 ttl=64 time=1.40 ms
64 bytes from 10.8.0.6: icmp_seq=2 ttl=64 time=1.44 ms
64 bytes from 10.8.0.6: icmp_seq=3 ttl=64 time=1.51 ms
^C
--- 10.8.0.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2013ms
rtt min/avg/max/mdev = 1.404/1.449/1.505/0.041 ms
carol:~#
```

Como podemos comprobar ahora se puede hacer ping desde Carol y Alice en ambas direcciones.

4.2 OpenVPN: Rendimiento

Para estas pruebas vamos a mantener solo el servicio activo en **Alice y Moon**, así evitamos modificar los ficheros en varios equipos. (Para parar el servicio en Carol usamos `ip`)

4.2.1 Rendimiento con las opciones por defecto

Para realizar las pruebas de rendimiento vamos a utilizar la herramienta `iperf`.

Pruebas desde su conexión directa

Alice → Moon:

- Moon (Servidor): `iperf -s`
- Alice (Cliente): `iperf -c 10.1.0.1 -i 1`

Conseguimos una velocidad de **6.28 Gbits/sec** en Alice y **6.27 Gbits/sec** en Moon

```
alice:~# iperf -c 10.1.0.1 -i 1
-----
Client connecting to 10.1.0.1, TCP port 5001
TCP window size: 85.0 KByte (default)
-----
[ 3] local 10.1.0.10 port 51740 connected with 10.1.0.1 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec  712 MBytes  5.97 Gbits/sec
[ 3] 1.0000-2.0000 sec  769 MBytes  6.45 Gbits/sec
[ 3] 2.0000-3.0000 sec  720 MBytes  6.04 Gbits/sec
[ 3] 3.0000-4.0000 sec  740 MBytes  6.21 Gbits/sec
[ 3] 4.0000-5.0000 sec  743 MBytes  6.23 Gbits/sec
[ 3] 5.0000-6.0000 sec  764 MBytes  6.40 Gbits/sec
[ 3] 6.0000-7.0000 sec  766 MBytes  6.43 Gbits/sec
^C[ 3] 7.0000-7.5493 sec  434 MBytes  6.63 Gbits/sec
[ 3] 0.0000-7.5493 sec  5.52 GBytes  6.28 Gbits/sec
alice:~#
```

```
moon:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.1.0.1 port 5001 connected with 10.1.0.10 port 51740
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-7.5500 sec  5.52 GBytes  6.27 Gbits/sec
moon:~#
```

Moon → Alice:

- Alice (Servidor): `iperf -s`
- Moon (Cliente): `iperf -c 10.1.0.10 -i 1`

Conseguimos una velocidad de **4.44 Gbits/sec** en Alice y **4.44 Gbits/sec** en Moon

```
alice:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.1.0.10 port 5001 connected with 10.1.0.1 port 44092
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-5.3776 sec  2.78 GBytes  4.44 Gbits/sec
alice:~#
```

```
moon:~# iperf -c 10.1.0.10 -i 1
-----
Client connecting to 10.1.0.10, TCP port 5001
TCP window size: 85.0 KByte (default)
-----
[ 3] local 10.1.0.1 port 44092 connected with 10.1.0.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec  507 MBytes  4.25 Gbits/sec
[ 3] 1.0000-2.0000 sec  530 MBytes  4.44 Gbits/sec
[ 3] 2.0000-3.0000 sec  526 MBytes  4.41 Gbits/sec
[ 3] 3.0000-4.0000 sec  518 MBytes  4.34 Gbits/sec
[ 3] 4.0000-5.0000 sec  576 MBytes  4.83 Gbits/sec
^C[ 3] 5.0000-5.3772 sec  188 MBytes  4.18 Gbits/sec
[ 3] 0.0000-5.3772 sec  2.78 GBytes  4.44 Gbits/sec
moon:~#
```

Pruebas usando el túnel

Alice → Moon:

- Moon (Servidor): `iperf -s`
- Alice (Cliente): `iperf -c 10.8.0.1 -i 1`

Conseguimos una velocidad de **113 Mbits/sec** en Alice y de **111 Mbits/sec** en Moon

```
alice:~# iperf -c 10.8.0.1 -i 1
-----
Client connecting to 10.8.0.1, TCP port 5001
TCP window size: 72.0 KByte (default)
-----
[ 3] local 10.8.0.6 port 37958 connected with 10.8.0.1 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec  14.4 MBytes  121 Mbits/sec
[ 3] 1.0000-2.0000 sec  13.0 MBytes  109 Mbits/sec
[ 3] 2.0000-3.0000 sec  13.8 MBytes  115 Mbits/sec
[ 3] 3.0000-4.0000 sec  13.4 MBytes  112 Mbits/sec
^C[ 3] 4.0000-4.8988 sec  11.4 MBytes  106 Mbits/sec
[ 3] 0.0000-4.8988 sec  65.9 MBytes  113 Mbits/sec
alice:~#
```

```
moon:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.1 port 5001 connected with 10.8.0.6 port 37958
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-4.9655 sec  65.8 MBytes  111 Mbits/sec
moon:~#
```

Moon → Alice:

- Alice (Servidor): `iperf -s`
- Moon (Cliente): `iperf -c 10.8.0.6 -i 1`

Conseguimos una velocidad de **109 Mbts/sec** en Alice y de **108 Mbts/sec** en Moon

```
alice:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.6 port 5001 connected with 10.8.0.1 port 37904
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-5.6597 sec 73.1 MBytes 108 Mbts/sec

moon:~# iperf -c 10.8.0.6 -i 1
Client connecting to 10.8.0.6, TCP port 5001
TCP window size: 72.0 KByte (default)
-----
[ 3] local 10.8.0.1 port 37904 connected with 10.8.0.6 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec 13.6 MBytes 114 Mbts/sec
[ 3] 1.0000-2.0000 sec 13.2 MBytes 111 Mbts/sec
[ 3] 2.0000-3.0000 sec 13.0 MBytes 109 Mbts/sec
[ 3] 3.0000-4.0000 sec 12.6 MBytes 106 Mbts/sec
[ 3] 4.0000-5.0000 sec 12.6 MBytes 106 Mbts/sec
^C[ 3] 5.0000-5.6238 sec 8.12 MBytes 109 Mbts/sec
[ 3] 0.0000-5.6238 sec 73.3 MBytes 109 Mbts/sec
moon:~#
```

Los resultados en la red directa (10.1.0.0/16) muestran capacidades de transferencia cercanas a los 6 Gbits/sec. Sin embargo, al realizar las pruebas a través de la red virtual de OpenVPN (10.8.0.0/24), el rendimiento cae drásticamente a valores cercanos a los 110 Mbts/sec. Esta penalización del ~98% es característica de las VPNs basadas en TLS que operan en el espacio de usuario con las opciones de configuración por defecto, lo que justifica la necesidad de aplicar técnicas de optimización de MTU y buffers en los siguientes apartados.

4.2.2 Mejora del rendimiento

Para consultar la MTU (Maximum Transfer Unit) de la interfaz TUN que utiliza OpenVPN por defecto usamos : `ifconfig tun0` (En Moon) . En nuestro caso es de **1500**

```
moon:~# ifconfig tun0
tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 1500
      inet 10.8.0.1 netmask 255.255.255.255 destination 10.8.0.2
      inet6 fe80::3683:f497:3ab1:57d1 prefixlen 64 scopeid 0x20
<link>
      unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txq
ueuelen 500 (UNSPEC)
      RX packets 79998 bytes 73106104 (69.7 MiB)
      RX errors 0 dropped 0 overruns 0 frame 0
      TX packets 82972 bytes 81031737 (77.2 MiB)
      TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Ahora vamos a incrementar ese MTU a **9000** (tamaño de una trama Jumbo Ethernet) y a deshabilitar la fragmentación realizada por OpenVPN, para ello modificamos el fichero

`/etc/openvpn/server.conf` en **moon** y `/etc/openvpn/client.conf` en **alice** añadiendo las siguientes directivas:

```
tun-mtu 9000
fragment 0      Desactiva la fragmentación de OpenVPN
mssfix 0        Desactiva el ajuste automático del MSS (TCP)
```

Después de aplicar estos cambios debemos reiniciar el servicio en ambos nodos. (`service openvpn restart`)

```
moon:~# ifconfig tun0
tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 9000
      inet 10.8.0.1 netmask 255.255.255.255 destination 10.8.0.
2
      inet6 fe80::b556:1843:3b19:b55a prefixlen 64 scopeid 0x20
<link>
      unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txq
ueuelen 500 (UNSPEC)
      RX packets 0 bytes 0 (0.0 B)
      RX errors 0 dropped 0 overruns 0 frame 0
      TX packets 4 bytes 304 (304.0 B)
      TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Ahora vamos a comprobar el rendimiento para este **MTU de 9000**:

Alice → Moon:

- Moon (Servidor): `iperf -s`
- Alice (Cliente): `iperf -c 10.8.0.1 -i 1`

Conseguimos una velocidad de **218 Mbts/sec** en Alice y de **216 Mbts/sec** en Moon

```
alice:~# iperf -c 10.8.0.1 -i 1
Client connecting to 10.8.0.1, TCP port 5001
TCP window size: 325 KByte (default)
-----
[ 3] local 10.8.0.6 port 34368 connected with 10.8.0.1 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec 25.9 MBytes 217 Mbts/sec
[ 3] 1.0000-2.0000 sec 23.5 MBytes 197 Mbts/sec
[ 3] 2.0000-3.0000 sec 24.6 MBytes 207 Mbts/sec
[ 3] 3.0000-4.0000 sec 26.8 MBytes 224 Mbts/sec
[ 3] 4.0000-5.0000 sec 28.0 MBytes 235 Mbts/sec
^C[ 3] 5.0000-5.5356 sec 14.9 MBytes 233 Mbts/sec
[ 3] 0.0000-5.5356 sec 144 MBytes 218 Mbts/sec
alice:~#
```

```
moon:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.1 port 5001 connected with 10.8.0.6 port 34368
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-5.5660 sec 144 MBytes 216 Mbts/sec
moon:~#
```

Moon → Alice:

- Alice (Servidor): `iperf -s`
- Moon (Cliente): `iperf -c 10.8.0.6 -i 1`

Conseguimos una velocidad de **224 Mbts/sec** en Alice y de **225 Mbts/sec** en Moon

```
alice:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.6 port 5001 connected with 10.8.0.1 port 48532
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-4.5658 sec 122 MBytes 224 Mbts/sec
alice:~#
```

```
moon:~# iperf -c 10.8.0.6 -i 1
Client connecting to 10.8.0.6, TCP port 5001
TCP window size: 325 KByte (default)
-----
[ 3] local 10.8.0.1 port 48532 connected with 10.8.0.6 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec 26.8 MBytes 224 Mbts/sec
[ 3] 1.0000-2.0000 sec 26.6 MBytes 223 Mbts/sec
[ 3] 2.0000-3.0000 sec 24.8 MBytes 208 Mbts/sec
[ 3] 3.0000-4.0000 sec 29.2 MBytes 245 Mbts/sec
^C[ 3] 4.0000-4.5461 sec 14.6 MBytes 225 Mbts/sec
[ 3] 0.0000-4.5461 sec 122 MBytes 225 Mbts/sec
moon:~#
```

Como podemos ver en los resultados se ha pasado de unos 110 Mbts/sec a **valores que superan los 220 Mbts/sec**, lo que supone **duplicar el rendimiento** del túnel simplemente ajustando el tamaño de la trama. Este incremento se debe a que, al procesar paquetes más grandes, la CPU tiene que manejar menos cabeceras y menos interrupciones para transferir la misma cantidad.

Ahora vamos a realizar unas pruebas cambiando el **MTU** por los siguientes valores **18000**, **36000**, **54000** y **65000** y mediremos el rendimiento.

- **MTU 18000**: Como podemos ver con este MTU se han conseguido velocidades de aproximadamente **324 Mbts/sec** (Siendo Alice cliente y Moon servidor) y de **305 Mbts/sec** (Siendo Alice servidor y Moon cliente)

```

alice:~# ip addr show tun0 | grep mtu
17: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER UP> mtu 18000 qdisc
pfifo_fast state UNKNOWN group default qlen 500
alice:~#

moon:~# ifconfig tun0
tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 18000
      inet 10.8.0.1 netmask 255.255.255.255 destination 10.8.0.
2
      inet6 fe80::d268:20e1:87a2:a8e prefixlen 64 scopeid 0x20<
link>
      unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txq
ueuelen 500 (UNSPEC)
      RX packets 0 bytes 0 (0.0 B)
      RX errors 0 dropped 0 overruns 0 frame 0
      TX packets 4 bytes 304 (304.0 B)
      TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

```

alice:~# iperf -c 10.8.0.1 -i 1
-----
Client connecting to 10.8.0.1, TCP port 5001
TCP window size: 645 KByte (default)
-----
[ 3] local 10.8.0.6 port 54578 connected with 10.8.0.1 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec 39.3 MBytes  329 Mbits/sec
[ 3] 1.0000-2.0000 sec 36.2 MBytes  304 Mbits/sec
[ 3] 2.0000-3.0000 sec 39.9 MBytes  334 Mbits/sec
^C[ 3] 3.0000-3.3709 sec 14.8 MBytes  334 Mbits/sec
[ 3] 0.0000-3.3709 sec 130 MBytes 324 Mbits/sec
alice:~#

moon:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.1 port 5001 connected with 10.8.0.6 port 54578
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-3.3818 sec 130 MBytes  322 Mbits/sec
moon:~#

```

```

alice:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.6 port 5001 connected with 10.8.0.1 port 43642
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-4.3929 sec 160 MBytes  305 Mbits/sec
alice:~#

moon:~# iperf -c 10.8.0.6 -i 1
-----
Client connecting to 10.8.0.6, TCP port 5001
TCP window size: 645 KByte (default)
-----
[ 3] local 10.8.0.1 port 43642 connected with 10.8.0.6 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec 39.3 MBytes  329 Mbits/sec
[ 3] 1.0000-2.0000 sec 38.5 MBytes  323 Mbits/sec
[ 3] 2.0000-3.0000 sec 35.6 MBytes  299 Mbits/sec
[ 3] 3.0000-4.0000 sec 33.4 MBytes  280 Mbits/sec
^C[ 3] 4.0000-4.3795 sec 12.9 MBytes  285 Mbits/sec
[ 3] 0.0000-4.3795 sec 160 MBytes 306 Mbits/sec
moon:~#

```

- **MTU 36000:** Como podemos ver con este MTU se han conseguido velocidades de aproximadamente **385 Mbits/sec** Siendo (Alice cliente y Moon servidor) y de **369 Mbits/sec** (Siendo Alice servidor y Moon cliente)

```

alice:~# ip addr show tun0 | grep mtu
21: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER UP> mtu 36000 qdisc
pfifo_fast state UNKNOWN group default qlen 500
alice:~#

moon:~# ifconfig tun0
tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 36000
      inet 10.8.0.1 netmask 255.255.255.255 destination 10.8.0.
2
      inet6 fe80::77d2:46d0:42ff:3645 prefixlen 64 scopeid 0x20
<link>
      unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txq
ueuelen 500 (UNSPEC)
      RX packets 13497 bytes 483440156 (461.0 MiB)
      RX errors 0 dropped 0 overruns 0 frame 0
      TX packets 6765 bytes 351964 (343.7 KiB)
      TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

```

alice:~# iperf -c 10.8.0.1 -i 1
-----
Client connecting to 10.8.0.1, TCP port 5001
TCP window size: 1.22 MByte (default)
-----
[ 3] local 10.8.0.6 port 35222 connected with 10.8.0.1 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec 50.0 MBytes  419 Mbits/sec
[ 3] 1.0000-2.0000 sec 47.0 MBytes  394 Mbits/sec
[ 3] 2.0000-3.0000 sec 48.5 MBytes  407 Mbits/sec
[ 3] 3.0000-4.0000 sec 48.0 MBytes  403 Mbits/sec
[ 3] 4.0000-5.0000 sec 40.4 MBytes  339 Mbits/sec
^C[ 3] 5.0000-5.4280 sec 15.5 MBytes  304 Mbits/sec
[ 3] 0.0000-5.4280 sec 249 MBytes 385 Mbits/sec
alice:~#

moon:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.1 port 5001 connected with 10.8.0.6 port 35222
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-5.4476 sec 249 MBytes  384 Mbits/sec
moon:~#

```

```

alice:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.6 port 5001 connected with 10.8.0.1 port 46028
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-6.3139 sec 277 MBytes  369 Mbits/sec
alice:~#

moon:~# iperf -c 10.8.0.6 -i 1
-----
Client connecting to 10.8.0.6, TCP port 5001
TCP window size: 853 KByte (default)
-----
[ 3] local 10.8.0.1 port 46028 connected with 10.8.0.6 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec 51.6 MBytes  433 Mbits/sec
[ 3] 1.0000-2.0000 sec 47.1 MBytes  395 Mbits/sec
[ 3] 2.0000-3.0000 sec 49.5 MBytes  415 Mbits/sec
[ 3] 3.0000-4.0000 sec 36.5 MBytes  306 Mbits/sec
[ 3] 4.0000-5.0000 sec 40.9 MBytes  343 Mbits/sec
[ 3] 5.0000-6.0000 sec 40.6 MBytes  341 Mbits/sec
^C[ 3] 6.0000-6.3001 sec 11.2 MBytes  314 Mbits/sec
[ 3] 0.0000-6.3001 sec 278 MBytes 369 Mbits/sec
moon:~#

```

- **MTU 54000:** Como podemos ver con este MTU se han conseguido velocidades de aproximadamente **465 Mbits/sec** (Siendo Alice cliente y Moon servidor) y de **445 Mbits/sec** (Siendo Alice servidor y Moon cliente)

```

alice:~# ip addr show tun0 | grep mtu
23: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 54000 qdisc
pfifo_fast state UNKNOWN group default qlen 500
alice:~# s

moon:~# ifconfig tun0
tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 54000
    inet 10.8.0.1 netmask 255.255.255.255 destination 10.8.0.
    2
    inet6 fe80::887b:b3ff:b7bd:5fd2 prefixlen 64 scopeid 0x20
    <link>
        unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txq
        ueuelen 500 (UNSPEC)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 4 bytes 304 (304.0 B)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

```

alice:~# iperf -c 10.8.0.1 -i 1
-----
Client connecting to 10.8.0.1, TCP port 5001
TCP window size: 1.22 MByte (default)
-----
[ 3] local 10.8.0.6 port 51564 connected with 10.8.0.1 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec 49.4 MBytes  414 Mbits/sec
[ 3] 1.0000-2.0000 sec 56.4 MBytes  473 Mbits/sec
[ 3] 2.0000-3.0000 sec 60.4 MBytes  506 Mbits/sec
[ 3] 3.0000-4.0000 sec 57.9 MBytes  485 Mbits/sec
^C[ 3] 4.0000-4.8041 sec 42.4 MBytes  442 Mbits/sec
[ 3] 0.0000-4.8041 sec 266 MBytes  465 Mbits/sec
alice:~#

moon:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.1 port 5001 connected with 10.8.0.6 port 51564
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-4.8132 sec 266 MBytes  464 Mbits/sec

```

```

alice:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.6 port 5001 connected with 10.8.0.1 port 54624
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-4.8510 sec 257 MBytes  444 Mbits/sec

moon:~# iperf -c 10.8.0.6 -i 1
-----
Client connecting to 10.8.0.6, TCP port 5001
TCP window size: 853 KByte (default)
-----
[ 3] local 10.8.0.1 port 54624 connected with 10.8.0.6 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec 59.3 MBytes  497 Mbits/sec
[ 3] 1.0000-2.0000 sec 50.5 MBytes  424 Mbits/sec
[ 3] 2.0000-3.0000 sec 49.5 MBytes  415 Mbits/sec
[ 3] 3.0000-4.0000 sec 50.6 MBytes  425 Mbits/sec
^C[ 3] 4.0000-4.8447 sec 47.1 MBytes  468 Mbits/sec
[ 3] 0.0000-4.8447 sec 257 MBytes  445 Mbits/sec

```

- **MTU 65000:** Como podemos ver con este MTU se han conseguido velocidades de aproximadamente **232 Mbits/sec** (Siendo Alice cliente y Moon servidor) y de **239 Mbits/sec** (Siendo Alice servidor y Moon cliente)

```

alice:~# ip addr show tun0 | grep mtu
27: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 65000 qdisc
pfifo_fast state UNKNOWN group default qlen 500
alice:~# s

moon:~# ifconfig tun0
tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 65000
    inet 10.8.0.1 netmask 255.255.255.255 destination 10.8.0.
    2
    inet6 fe80::ef13:9774:e487:53f5 prefixlen 64 scopeid 0x20
    <link>
        unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txq
        ueuelen 500 (UNSPEC)
        RX packets 3076 bytes 144601432 (137.9 MiB)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 2903 bytes 106777684 (101.8 MiB)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

```

alice:~# iperf -c 10.8.0.1 -i 1
-----
Client connecting to 10.8.0.1, TCP port 5001
TCP window size: 1.22 MByte (default)
-----
[ 3] local 10.8.0.6 port 46414 connected with 10.8.0.1 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec 27.4 MBytes  230 Mbits/sec
[ 3] 1.0000-2.0000 sec 28.2 MBytes  237 Mbits/sec
[ 3] 2.0000-3.0000 sec 27.8 MBytes  233 Mbits/sec
[ 3] 3.0000-4.0000 sec 27.8 MBytes  233 Mbits/sec
^C[ 3] 4.0000-4.9741 sec 26.8 MBytes  230 Mbits/sec
[ 3] 0.0000-4.9741 sec 138 MBytes  233 Mbits/sec
alice:~#

moon:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.1 port 5001 connected with 10.8.0.6 port 46414
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-4.9993 sec 138 MBytes  231 Mbits/sec

```

```

alice:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.6 port 5001 connected with 10.8.0.1 port 50662
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-3.5831 sec 102 MBytes  238 Mbits/sec

moon:~# iperf -c 10.8.0.6 -i 1
-----
Client connecting to 10.8.0.6, TCP port 5001
TCP window size: 853 KByte (default)
-----
[ 3] local 10.8.0.1 port 50662 connected with 10.8.0.6 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec 28.0 MBytes  235 Mbits/sec
[ 3] 1.0000-2.0000 sec 28.1 MBytes  236 Mbits/sec
[ 3] 2.0000-3.0000 sec 28.8 MBytes  241 Mbits/sec
^C[ 3] 3.0000-3.5698 sec 16.9 MBytes  240 Mbits/sec
[ 3] 0.0000-3.5698 sec 102 MBytes  239 Mbits/sec

```

Conclusiones:

- **Incremento de velocidad:** A medida que vamos aumentando el MTU desde 18000 hasta 54000, el rendimiento del túnel mejora de forma constante. Esto se debe a que el sistema procesa tramas más grandes, lo que reduce enormemente el número de paquetes totales y, por

ende, disminuye las interrupciones de la CPU y la sobrecarga de las cabeceras por cada byte transmitido.

- **MTU óptima:** Como hemos podido observar en las mediciones anteriores, la **MTU óptima es de 54000**. Con este valor se han alcanzado unas velocidades de 465 Mbits/sec, es decir se ha cuadruplicado la velocidad inicial de 110 Mbits/sec (Con MTU de 1500).
- **Degradación en 65000:** Al fijar una MTU de 65000, observamos una caída en el rendimiento (Bajando a unos 230 Mbits/sec). Esto ocurre porque nos aproximamos al límite teórico de un paquete IP (65535 bytes). Al sumar las cabeceras adicionales de la VPN y el cifrado, el paquete final excede el tamaño máximo permitido, provocando inestabilidad en el túnel o forzando al sistema operativo a realizar una fragmentación ineficiente que consume recursos adicionales de la CPU.

Ahora vamos a realizar unas pruebas con los siguientes **MTU 1400, 1300 y 1200**. Después de realizar los cambios iremos midiendo el rendimiento de cada MTU.

- **MTU de 1400:** Como podemos ver con este MTU se han conseguido velocidades de aproximadamente **15 Mbits/sec** (Siendo Alice cliente y Moon servidor) y de **17 Mbits/sec** (Siendo Alice servidor y Moon cliente)

```
alice:~# ip addr show tun0 | grep mtu
28: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1400 qdisc
pfifo_fast state UNKNOWN group default qlen 500
alice:~#
```

```
moon:~# ifconfig tun0 | grep mtu
tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 1400
moon:~#
```

```
alice:~# iperf -c 10.8.0.1 -i 1
-----
Client connecting to 10.8.0.1, TCP port 5001
TCP window size: 45.0 KByte (default)
-----
[ 3] local 10.8.0.6 port 50212 connected with 10.8.0.1 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec  2.00 MBytes 16.8 Mbits/sec
[ 3] 1.0000-2.0000 sec  1.88 MBytes 15.7 Mbits/sec
[ 3] 2.0000-3.0000 sec  1.62 MBytes 13.6 Mbits/sec
[ 3] 3.0000-4.0000 sec  1.75 MBytes 14.7 Mbits/sec
^C[ 3] 4.0000-4.5932 sec  1.25 MBytes 17.7 Mbits/sec
[ 3] 0.0000-4.5932 sec  8.50 MBytes 15.5 Mbits/sec
alice:~#
```

```
moon:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.1 port 5001 connected with 10.8.0.6 port 50212
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-4.7452 sec  8.38 MBytes 14.8 Mbits/sec
moon:~#
```

```
alice:~# iperf -s
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.6 port 5001 connected with 10.8.0.1 port 36158
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-3.8443 sec  7.50 MBytes 16.4 Mbits/sec
alice:~#
```

```
moon:~# iperf -c 10.8.0.6 -i 1
-----
Client connecting to 10.8.0.6, TCP port 5001
TCP window size: 45.0 KByte (default)
-----
[ 3] local 10.8.0.1 port 36158 connected with 10.8.0.6 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec  2.13 MBytes 17.8 Mbits/sec
[ 3] 1.0000-2.0000 sec  2.12 MBytes 17.8 Mbits/sec
[ 3] 2.0000-3.0000 sec  1.88 MBytes 15.7 Mbits/sec
^C[ 3] 3.0000-3.7143 sec  1.50 MBytes 17.6 Mbits/sec
[ 3] 0.0000-3.7143 sec  7.63 MBytes 17.2 Mbits/sec
moon:~#
```

- **MTU DE 1300:** Como podemos ver con este MTU se han conseguido velocidades de aproximadamente **13 Mbits/sec** (Siendo Alice cliente y Moon servidor) y de **15 Mbits/sec** (Siendo Alice servidor y Moon cliente)

```
alice:~# ip addr show tun0 | grep mtu
29: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1300 qdisc
pfifo_fast state UNKNOWN group default qlen 500
alice:~#
```

```
moon:~# ifconfig tun0 | grep mtu
tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 1300
moon:~#
```

```

alice:~# iperf -c 10.8.0.1 -i 1
Client connecting to 10.8.0.1, TCP port 5001
TCP window size: 45.0 KByte (default)
-----
[ 3] local 10.8.0.6 port 48868 connected with 10.8.0.1 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec  1.75 MBytes 14.7 Mbits/sec
[ 3] 1.0000-2.0000 sec  1.38 MBytes 11.5 Mbits/sec
[ 3] 2.0000-3.0000 sec  2.00 MBytes 16.8 Mbits/sec
[ 3] 3.0000-4.0000 sec  1.62 MBytes 13.6 Mbits/sec
^C[ 3] 4.0000-4.3969 sec   640 KBytes 13.2 Mbits/sec
[ 3] 0.0000-4.3969 sec  7.38 MBytes 14.1 Mbits/sec
alice:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.6 port 5001 connected with 10.8.0.1 port 57294
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-3.6903 sec  6.25 MBytes 14.2 Mbits/sec

moon:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.1 port 5001 connected with 10.8.0.6 port 48868
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-4.7129 sec  7.25 MBytes 12.9 Mbits/sec
^Cmoon:~# iperf -c 10.8.0.6 -i 1
Client connecting to 10.8.0.6, TCP port 5001
TCP window size: 63.0 KByte (default)
-----
[ 3] local 10.8.0.1 port 57294 connected with 10.8.0.6 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec  1.88 MBytes 15.7 Mbits/sec
[ 3] 1.0000-2.0000 sec  1.88 MBytes 15.7 Mbits/sec
[ 3] 2.0000-3.0000 sec  1.75 MBytes 14.7 Mbits/sec
^C[ 3] 3.0000-3.4701 sec   896 KBytes 15.6 Mbits/sec
[ 3] 0.0000-3.4701 sec  6.38 MBytes 15.4 Mbits/sec
moon:~# S

```

- **MTU DE 1200:** Como podemos ver con este MTU se han conseguido velocidades de aproximadamente **13 Mbits/sec** (Siendo Alice cliente y Moon servidor) y de **14 Mbits/sec** (Siendo Alice servidor y Moon cliente)

```

alice:~# ip addr show tun0 | grep mtu
tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1200 qdisc pfifo_fast state UNKNOWN group default qlen 500
alice:~#

moon:~# ifconfig tun0 | grep mtu
tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 1200
moon:~#

alice:~# iperf -c 10.8.0.1 -i 1
Client connecting to 10.8.0.1, TCP port 5001
TCP window size: 45.0 KByte (default)
-----
[ 3] local 10.8.0.6 port 56796 connected with 10.8.0.1 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec  1.75 MBytes 14.7 Mbits/sec
[ 3] 1.0000-2.0000 sec  1.62 MBytes 13.6 Mbits/sec
[ 3] 2.0000-3.0000 sec  1.75 MBytes 14.7 Mbits/sec
[ 3] 3.0000-4.0000 sec  1.50 MBytes 12.6 Mbits/sec
^C[ 3] 4.0000-4.3409 sec   640 KBytes 15.4 Mbits/sec
[ 3] 0.0000-4.3409 sec  7.25 MBytes 14.0 Mbits/sec
alice:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.6 port 5001 connected with 10.8.0.1 port 35200
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-3.5583 sec  5.50 MBytes 13.0 Mbits/sec

moon:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 4] local 10.8.0.1 port 5001 connected with 10.8.0.6 port 56796
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0000-4.8421 sec  7.13 MBytes 12.3 Mbits/sec
^Cmoon:~# iperf -c 10.8.0.6 -i 1
Client connecting to 10.8.0.6, TCP port 5001
TCP window size: 45.0 KByte (default)
-----
[ 3] local 10.8.0.1 port 35200 connected with 10.8.0.6 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0000-1.0000 sec  1.75 MBytes 14.7 Mbits/sec
[ 3] 1.0000-2.0000 sec  1.62 MBytes 13.6 Mbits/sec
[ 3] 2.0000-3.0000 sec  1.75 MBytes 14.7 Mbits/sec
^C[ 3] 3.0000-3.3056 sec   512 KBytes 13.7 Mbits/sec
[ 3] 0.0000-3.3056 sec  5.63 MBytes 14.3 Mbits/sec
moon:~#

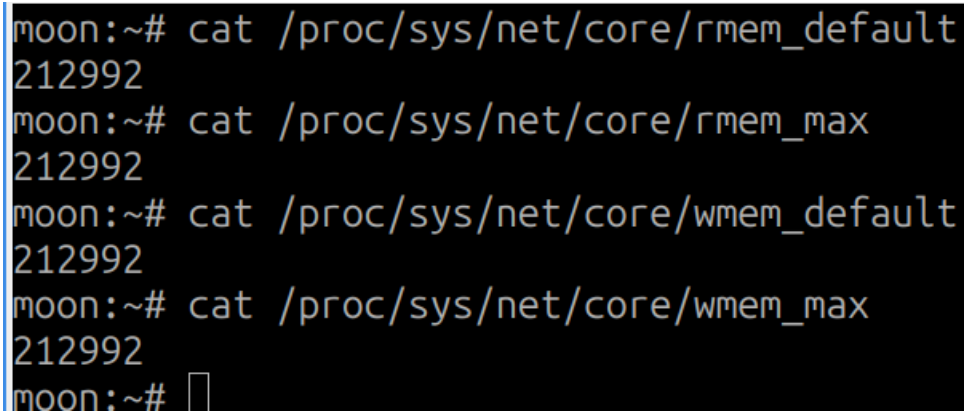
```

Conclusiones:

- Al reducir el tamaño de la MTU, la proporción de "datos útiles" frente a las cabeceras (IP, UDP y las propias de OpenVPN) disminuye. Para enviar la misma cantidad de información, el sistema debe generar muchísimos más paquetes, lo que desperdicia gran parte del ancho de banda en transmitir metadatos repetidos.
- OpenVPN funciona en el espacio de usuario. Cada paquete enviado o recibido requiere una interrupción del sistema y un cambio de contexto entre el kernel y la aplicación. Al usar MTUs bajas (como 1200), el número de paquetes por segundo se dispara, haciendo que la CPU dedique la mayor parte de su tiempo a gestionar el tráfico y el cifrado de miles de pequeños bloques en lugar de procesar la transferencia de datos real.
- Los algoritmos de cifrado como **AES-128-CBC** trabajan de forma más eficiente con bloques de datos más grandes. Al fragmentar la información en paquetes tan pequeños, el coste computacional de cifrar y autenticar cada paquete individual penaliza severamente la velocidad final.

Ahora vamos a aplicar la optimización de los buffers de envío y recepción. El objetivo es evitar que el flujo de datos se detenga por falta de espacio en la memoria intermedia del sistema operativo. Antes de realizar cambios, comprobamos cuáles son los valores por defecto y los límites máximos permitidos por el kernel de Linux:

```
moon$ cat /proc/sys/net/core/rmem_default # Valor por defecto de recepción
moon$ cat /proc/sys/net/core/rmem_max     # Valor máximo de recepción
moon$ cat /proc/sys/net/core/wmem_default # Valor por defecto de envío
moon$ cat /proc/sys/net/core/wmem_max     # Valor máximo de envío
```



```
moon:~# cat /proc/sys/net/core/rmem_default
212992
moon:~# cat /proc/sys/net/core/rmem_max
212992
moon:~# cat /proc/sys/net/core/wmem_default
212992
moon:~# cat /proc/sys/net/core/wmem_max
212992
moon:~#
```

Editamos el fichero `/etc/openvpn/server.conf` en **moon** para aplicar las nuevas directivas. Utilizaremos el valor `0` para que el servidor delegue la gestión en el SO, y usaremos directivas `push` para que los clientes (como Alice) configuren sus buffers a **512 KB** de forma automática.

```
# Añadir al final de /etc/openvpn/server.conf
sndbuf 0
rcvbuf 0
push "sndbuf 524288"
push "rcvbuf 524288"
```

Si la MTU fuese 54000 y sus buffers muy pequeños habría una alta pérdida de paquetes y velocidad de transferencia muy inestable en iperf.

Teóricamente, al aplicar esta configuración el servidor OpenVPN delega la gestión de los buffers de envío y recepción al sistema operativo, permitiendo que el kernel ajuste dinámicamente su tamaño según la carga. Al mismo tiempo, los clientes reciben automáticamente una configuración de buffers de 512 KB, lo que evita que el flujo de datos se detenga por falta de espacio en la memoria intermedia. Como resultado, se espera una mejora en el rendimiento y la estabilidad de la VPN, con un flujo de datos más continuo y una menor probabilidad de pérdidas de paquetes, especialmente en situaciones de alto tráfico o latencia.

Mide el rendimiento variando las opciones de encriptado y compresión. ¿Qué conclusiones extraes?

El cifrado es el factor que más influye en el rendimiento del túnel.

- **Costo de CPU:** OpenVPN es una aplicación monohilo; esto significa que solo puede utilizar un núcleo del procesador para cifrar y descifrar el tráfico.
- **AES-256-GCM vs AES-128-CBC:** Al pasar de 128 a 256 bits, la carga de trabajo de la CPU aumenta considerablemente.
- **Cipher None:** Al eliminar el cifrado, la velocidad se dispara hacia el límite físico de la red virtual, ya que la CPU solo se encarga del encapsulamiento.

La compresión `LZO` busca reducir el tamaño de los paquetes antes de enviarlos.

- **Efecto en iperf:** Dado que `iperf` genera tráfico de datos aleatorios o ya optimizados, la compresión no logra reducir el tamaño del paquete.
- **Conclusión:** En redes de alta velocidad, la compresión suele ser contraproducente. Añade un retardo adicional de procesamiento (latencia) y consumo de CPU sin aportar una reducción real en el volumen de datos transferidos.

4.2.3 NIC offloading

Para conocer el límite máximo de rendimiento que podemos alcanzar y compararlo de forma justa con OpenVPN, es necesario desactivar las funciones de **offloading** de las interfaces físicas. Las interfaces virtuales `tun` no implementan estas mejoras, por lo que desactivarlas en la red física nos da una "línea base" real del rendimiento de nuestra CPU gestionando paquetes.

Primero, mostramos las opciones actuales del adaptador `eth1` en **moon** para ver qué funciones están aceleradas por hardware:

```
moon$ ethtool -k eth1
```

El NIC Offloading es como un "asistente" de la tarjeta de red que libera de trabajo a la CPU. Sin embargo, en el taller aprendes que, al usar una VPN, este asistente queda "ciego" por culpa del cifrado, obligando a la CPU a hacerlo todo y reduciendo el rendimiento.

El NIC Offloading funciona muy bien con tráfico normal, pero falla con las VPN:

Como los datos van cifrados y encapsulados, la tarjeta de red no sabe qué hay dentro.

Al no entender el contenido, la tarjeta de red no puede ayudar a "trocear" o "acelerar" el tráfico de forma automática.

Resultado: Todo el trabajo pesado recae en la CPU, y por eso la velocidad cae de 6 Gbits/s a 100-500 Mbits/s.

```

moon:~# ethtool -k eth1
Features for eth1:
rx-checksumming: on [fixed]
tx-checksumming: off
    tx-checksum-ipv4: off [fixed]
    tx-checksum-ip-generic: off
    tx-checksum-ipv6: off [fixed]
    tx-checksum-fcoe-crc: off [fixed]
    tx-checksum-sctp: off [fixed]
scatter-gather: on
    tx-scatter-gather: on
    tx-scatter-gather-fraglist: off [fixed]
tcp-segmentation-offload: off
    tx-tcp-segmentation: off [requested on]
    tx-tcp-ecn-segmentation: off [requested on]
    tx-tcp-mangleid-segmentation: off
    tx-tcp6-segmentation: off [requested on]
generic-segmentation-offload: on
generic-receive-offload: on
large-receive-offload: off [fixed]
rx-vlan-offload: off [fixed]
tx-vlan-offload: off [fixed]
ntuple-filters: off [fixed]
receive-hashing: off [fixed]
highdma: on [fixed]

```

Procedemos a desactivar el offloading en ambos extremos para que la CPU asuma toda la carga de segmentación y procesamiento de paquetes:

```
moon$ ethtool -K eth1 tx off rx off gso off gro off
```

```
alice$ ethtool -K eth0 tx off rx off gso off gro off
```

Medimos la velocidad en la red directa (`10.1.0.0/16`) para ver el impacto. Estos valores representan el máximo absoluto que la CPU puede mover sin el "extra" de trabajo que supone el cifrado de la VPN.

- **Rendimiento Alice → Moon:**

- Servidor (Moon): `iperf -s`
- Cliente (Alice): `iperf -c 10.1.0.1 -i 1`
- **Resultado:** 184 Mb/s

```

alice:~# iperf -c 10.1.0.1 -i 1
Client connecting to 10.1.0.1, TCP port 5001
TCP window size: 196 KByte (default)
[ 3] local 10.1.0.10 port 44064 connected with 10.1.0.1 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec  22.3 MBytes  187 Mbits/sec
[ 3] 1.0000-2.0000 sec  21.5 MBytes  180 Mbits/sec
[ 3] 2.0000-3.0000 sec  21.8 MBytes  182 Mbits/sec
[ 3] 3.0000-4.0000 sec  22.1 MBytes  186 Mbits/sec
[ 3] 4.0000-5.0000 sec  22.0 MBytes  185 Mbits/sec
^C[ 3] 5.0000-5.5908 sec  13.0 MBytes  185 Mbits/sec
[ 3] 0.0000-5.5908 sec  123 MBytes  184 Mbits/sec
alice:~# s

moon:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
[ 4] local 10.1.0.1 port 5001 connected with 10.1.0.10 port 44064
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-5.6259 sec  123 MBytes  183 Mbits/sec

```

• Rendimiento Moon → Alice:

- Servidor (Alice): `iperf -s`
- Cliente (Moon): `iperf -c 10.1.0.10 -i 1`
- **Resultado:** 183 Mbits/sec

```

alice:~# iperf -s
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
[ 4] local 10.1.0.10 port 5001 connected with 10.1.0.1 port 60356
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-3.8453 sec  83.1 MBytes  181 Mbits/sec

moon:~# iperf -c 10.1.0.10 -i 1
Client connecting to 10.1.0.10, TCP port 5001
TCP window size: 187 KByte (default)
[ 3] local 10.1.0.1 port 60356 connected with 10.1.0.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-1.0000 sec  21.5 MBytes  180 Mbits/sec
[ 3] 1.0000-2.0000 sec  21.9 MBytes  184 Mbits/sec
[ 3] 2.0000-3.0000 sec  21.5 MBytes  180 Mbits/sec
^C[ 3] 3.0000-3.8245 sec  18.4 MBytes  187 Mbits/sec
[ 3] 0.0000-3.8245 sec  83.3 MBytes  183 Mbits/sec
moon:~# Ss

```

Una vez finalizada la comparativa, restauramos el offloading para devolver la eficiencia al sistema:

```

moon$ ethtool -K eth1 tx on rx on gso on gro on
alice$ ethtool -K eth0 tx on rx on gso on gro on

```

Al desactivar el offloading, todo el trabajo de segmentación y procesamiento de paquetes pasa a la CPU, que antes realizaba la tarjeta de red. Esto incrementa mucho el número de paquetes que la CPU debe gestionar y la convierte en el principal cuello de botella. Por eso, el rendimiento cae de valores muy altos (como 6 Gbit/s, que dependen de aceleración por hardware) a unos 180 Mbit/s, que representan el límite real que la CPU puede manejar sin offloading y sin cifrado.

1. Apartado 3.1: Escenario Roadwarrior (Carol a Moon)

Este es el escenario de "teletrabajo".

- Quién conecta: Un host remoto (Carol) con un Gateway (Moon).
- Modo: Modo Túnel. Se crea un sobre nuevo para que Carol pueda entrar en la red interna (10.1.0.0/16) donde está Alice.
- Cifrado: Es NULL (nulo). Es la diferencia más importante. Se hace así para que puedas usar tcpdump y ver que, aunque hay un túnel IPsec, los datos (el "hola" de un ping) aún son legibles. Sirve para verificar que la "tubería" está bien conectada.
- Roles: Carol es la Iniciadora y Moon es el Responder.

2. Apartado 3.2: Escenario Host-to-Host (Moon a Sun)

Aquí la conexión es entre los dos "jefes" de las redes.

- Quién conecta: Gateway (Moon) con Gateway (Sun).
- Modo: Puede configurarse como Modo Transporte (para proteger solo el tráfico entre ellos) o Modo Túnel.
- Cifrado: Aquí ya se usa AES-GCM (cifrado real y fuerte). Si intentas cotillear el tráfico con tcpdump, solo verás basura cifrada.
- Firewall: En este apartado se introducen reglas de iptables más estrictas. Se bloquea todo el tráfico y solo se permite el que pase específicamente por las políticas de IPsec.

3. Apartado 3.3: Escenario Site-to-Site (Red a Red)

Este es el nivel más alto, donde conectas dos oficinas completas.

- Quién conecta: La LAN de Alice (10.1.x.x) con la LAN de Bob (10.2.x.x).
- El túnel: Los Gateways (Moon y Sun) mantienen el túnel abierto de forma permanente.
- Transparencia: Para Alice y Bob, el túnel es invisible. Ellos simplemente hacen ping a la IP del otro y los Gateways se encargan de cifrar, enviar y descifrar automáticamente.

Diferencia con el 3.1: En el 3.1 solo un usuario (Carol) entraba a una red. Aquí, dos redes completas se fusionan de forma segura a través de Internet